

Práctica 2

Teatro Escuela Carlos Mayolo

Programación Orientada a Objetos

Grupo 2

Equipo 5

Integrantes

Francisco José Cerén Porto

Juan Pablo Mira Cañas

Miguel Vélez Bernal

Danna Valeria Pérez Niño

Oscar David Arango Garcia

Profesor

Jaime Alberto Guzmán Luna

Universidad Nacional de Colombia - Sede Medellín

2025

Descripción General

- El programa está diseñado para manejar la logística de un teatro, en las que se simplifican tareas como, vender tiquetes , organizar las funciones a presentar , gestionar a los empleados y las clases de teatro, todo esto diseñado para que la interacción con el usuario sea lo más sencillo posible, ya que implementa diferentes herramientas gráficas como tablas o incluso la implementación de colores para que sea más intuitivo el comportamiento del mismo programa
- El programa al tener que trabajar con clientes debe procurar administrar y manipular toda la información asociada a ellos, como su identificador , y demás información que pueda ir acumulando durante la interacción con el mismo programa, para eso se prevé una especie de inicio de sesión donde se busca identificar a el cliente para luego saber a que se le va asociar toda la información, como compras y de más detalles
- Pretendemos abordar diversos problemas relacionados con la gestión logística y operativa de un teatro, buscando optimizar tanto las tareas administrativas como la experiencia del usuario.
- - Las problemáticas comunes a las que un teatro debe enfrentar se son comúnmente.
 - - Manejo de clientes

El manejo de clientes falla cuando no se tiene un correcto almacenamiento de los datos asociados, como por ejemplo compras dentro del teatro; mantener un buen manejo de la información permitirá no solo ser más eficaces, sino también prestar un mejor servicio ya que conociendo la información del cliente podemos ofrecer descuentos teniendo en cuenta sus diferentes interacciones.

■ Pago de nómina

En cualquier industria el manejo del dinero siempre puede resultar en un problemática ya que el manejo de esta sin una correcta planeación puede no resultar como debería, lo que el programa implementa es una correlación con la venta de tiquetes donde todo el dinero recaudado va directamente a tesorería donde espera ser distribuido a los trabajadores dependiendo de la cantidad de trabajo hecho, haciéndolo casi que en automático.

■ Manejo de Obras

Las funciones presentadas son casi que lo más importante de un teatro, es por eso que administrar el cómo se programan durante la semana puede resultar de una tarea complicada, es por eso que lo que hacemos es que el mismo administrador puede asignar de manera fácil una función a una sala junto con la obra que desee , siempre asegurándose de que esta esté realmente disponible

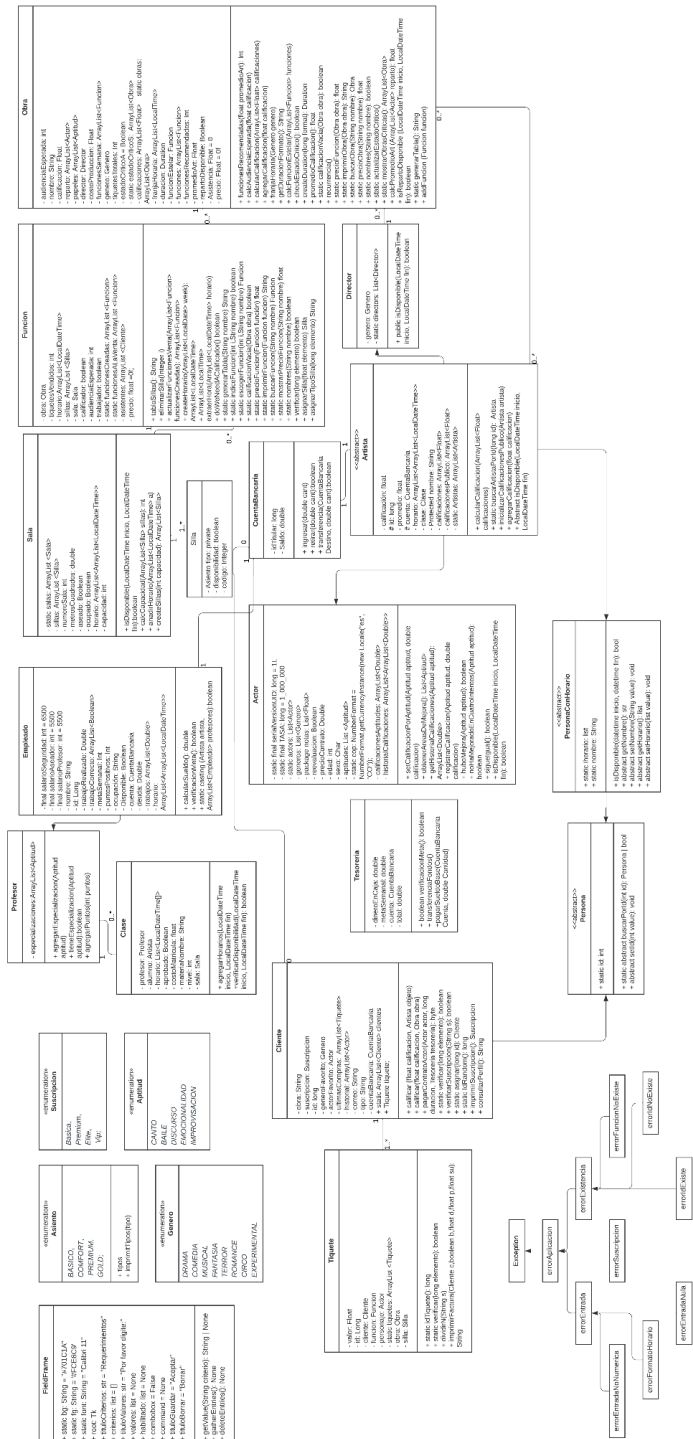
■ Manejo de Trabajadores

Contratar actores y asociarlos a clases puede ser algo tedioso , es por eso que lo automatizamos a una manera que simplemente necesite de la información básica para crear toda la interacción

El programa fue diseñado para cumplir con estos requisitos:

- Se debe poder manejar la venta de tiquetes, incluyendo venta de funciones, vista de asientos, generación de facturas y descuentos por suscripción
- Capacidad de administrar lo concerniente con los actores, permitiendo la contratación de sus servicios en horarios libres y su asignación para trabajar en funciones, además del uso de sus calificaciones para implementar lógicas de ordenamiento.
- Se incluye la creación de obras junto a sus respectivos directores, y la creación de nuevas funciones a partir de obras preexistentes, la asignación de actores a obras y el registro de su nombre y género principal.
- Implementar el uso interconectado de horarios, que evite que eventos como las funciones y la limpieza de salas ocurran simultáneamente, y evita conflictos de horarios en actores y empleados.
- Gestión de personal, incluyendo la contratación y despido del empleado (incluyendo el tipo Seguridad, Aseador y Profesor)
- Asignar clases a los actores para mejorar sus aptitudes, guiados por profesores que hagan parte del personal de teatro. Su desarrollo será determinado por las calificaciones que reciba de los profesores evaluadores.
- Persistencia de la información almacenada por el teatro para mantener los cambios recientes y el historial de ventas, horarios, actores contratados, entre otros.
- Mantener estabilidad, al tener un diseño de encapsulamiento privado, el manejo de excepciones y el uso de constantes, se evita la posible interferencia externa con datos inválidos, al mismo tiempo que se mantiene la posibilidad de manejar entradas de tipo válido pero erróneas provenientes del usuario.
- La interfaz de usuario resulta intuitiva y fácil de usar y entender, acompañada de colores que faciliten el entendimiento de mensajes de éxito, error o sugerencia.
- Centralización de los datos, para evitar que existan múltiples copias que puedan causar errores o comportamientos indeseados.

UML



https://lucid.app/lucidchart/d1e59da6-0bec-457a-bffe-024996cc153f/edit?viewport_loc=551%2C-392%2C2195%2C1479%2CHWEp-vi-RSFO&invitationId=inv_8e9ed224-8432-4f89-b8d0-594020442bb7

Descripción de la implementación de características de programación orientada a objetos en el proyecto.

- **Clases Abstractas (1 caso) y Métodos Abstractos (1 caso) e Interfaz:**

```
fmcscp, last week | 1 author (fmcscp)
from __future__ import annotations    fmcscp, last week + add: interfaz Persona, compatibilidad con an
from abc import ABC, abstractmethod

fmcscp, last week | 1 author (fmcscp)
class Persona(ABC):

    id: int

    #método estático-abstracto
    # estático para ser llamado desde la subclase,
    # abstracto para definir la lista donde se buscará la identificación
    @staticmethod
    @abstractmethod
    def buscarPorId(cls, id: int) -> Persona | bool:
        """Busca en la lista de instancia Teatro respectiva, si el número de identificación existe"""
        pass

    #métodos abstractos
    @abstractmethod
    def getId(self) -> int:
        pass

    @abstractmethod
    def setId(self, value: int) -> None:
        pass
```

Clase abstracta: Persona.py línea 4

Método abstracto: Persona.py línea 13

La clase abstracta Persona define el método abstracto buscarPorId(cls, id: int) obligando a sus subclases (conHorario) a implementar su propia lógica búsqueda

Como conHorario no implementa su propia lógica de búsqueda, esta le hereda esa tarea a su hijo Artista, el cual verifica que el id que el usuario digite coincida con alguno id perteneciente a los artistas que se encuentran registrados en nuestra base de datos

- **Herencia (1 caso):**

```
class Artista(Persona, conHorario):
```

```
class Actor(Artista):
```

```
class Director(Artista):
```

Clase padre: Artista.py

Clases hijas: Actor.py y Director.py

La herencia en este proyecto permite reutilizar código y establecer una jerarquía lógica entre Artista y sus subclases (Actor y Director).

Artista define atributos y métodos comunes como id, nombre, calificacion, y isDisponible(), evitando redundancia. Actor y Director heredan estas características pero implementan su propia lógica específica, como la gestión de horarios o calificaciones.

Esto mejora la organización del código, facilita la escalabilidad y permite manejar a los artistas de forma general sin perder sus particularidades.

• Ligadura dinámica (2 casos) asociadas al modelo lógico de la aplicación.

1er caso

```
actorsForRental = list(filter(lambda actor: actor.isDisponible(fechaInicio, fechaFin), actorsForRental))
```

Main.py linea 3491

```
def isDisponible(self, inicio: datetime, fin: datetime) -> bool:
    """Revisa en cada intervalo de horarios guardados si el nuevo horario se solapa"""
    for evento in self.getHorario():
        if ((inicio < evento[1]) and (fin > evento[0])):
            return False
    return True
```

conHorario.py linea 11

Ligadura dinámica:

Se resuelve por ligadura dinámica porque el método isDisponible, de actor, es de instancia.

2do caso

```
if emp.getMetaSemanal() == base:
    cant_trabajadores_principiantes += 1
```

Main.py linea 1708

```
def getMetaSemanal(self):
    return self.__metaSemanal
```

Empleado.py linea 94

Se resuelve por ligadura dinámica porque el método getMetaSemanal es de instancia.

- **Atributos de clase (1 caso) y métodos de clase (1 caso):**

```
class FieldFrame(Frame):  
    bg = "#701C1A"  
    fg = '#FCE6C9'  
    font = "Calibri 11"  
    fieldTest = False
```

FieldFrame.py

```
@classmethod  
def buscarPorId(cls, id: int) -> Artista | None:  
    for artista in Teatro.getInstancia().getArtistas():  
        if (artista.getId() == id):  
            return artista  
    return None
```

Artista.py linea 30

El atributo es porque este tiene en cuenta todas las instancias de la clase FieldFrame, de este modo todas comparten su interacción con el atributo. El método es estático porque este interactúa con la lista estática de los artistas, para que siempre que se ejecute el método, no hayan cambios en las listas de artistas, ya que siempre son los mismos.

- **Uso de constante (1 caso)**

```
SALARIOSEGURIDAD = 6500  
SALARIOASEADOR = 5500  
SALARIOPROFESOR = 5500
```

Empleado.py linea 5-7

En este caso los salarios de los empleados son constantes porque siempre son el mismo.

- **Encapsulamiento (private y public)**

```
self.genero = genero
self.horario = []
```

Director.py linea 6,7

```
self.__nombre = nombre
self.__id = id
```

Artista.py linea 14,15

Se definieron los atributos genero y horario como públicos, para que sean accesibles desde cualquier parte del código y así el director pueda cambiar su genero favorito en cualquier momento y su horario sea más flexible, en cambio el nombre e Id son aspectos únicos que le pertenecen al director

Inicializadores:

```
def __init__(self, nombre, ID, Ocupacion):
    self._nombre = nombre
    self._id = ID
    self._ocupacion = Ocupacion
    self._cuenta = CuentaBancaria(ID, 0)
    self._metaSemanal = 6
    self._puntosPositivos = 0
    self._disponible = True
    self._deuda = 0
    self._horario = []
    self._trabajoRealizado = 0
    self._trabajoCorrecto = []
    self._trabajos = []
```

Empleado.py


```

obras = []
def __init__(self, audienciaEsperada = 0, nombre = "", calificacion = 0, reparto = [], papeles = [], director = None, costoProduccion = 0, funcionesSemana = 0, funcionesRecomendadas = 0, promedioArt = 0, repartoDisponible = 0, asistencia = 0, precio = 0):
    self.__audienciaEsperada = audienciaEsperada
    self.__nombre = nombre
    self.__calificacion = calificacion
    self.__reparto = reparto
    self.__papeles = papeles
    self.__director = director
    self.__costoProduccion = costoProduccion
    self.__funcionesSemana = funcionesSemana
    self.__genero = genero
    self.__tiquetesTotales = tiquetesTotales
    self.__estadoCriticoA = estadoCriticoA
    self.__calificaciones = calificaciones
    self.__franjaHoraria = franjaHoraria
    self.__duracion = duracion
    self.__funcionEstelar = funcionEstelar
    self.__funciones = funciones
    self.__funcionesRecomendadas = funcionesRecomendadas
    self.__promedioArt = promedioArt
    self.__repartoDisponible = repartoDisponible
    self.__asistencia = asistencia
    self.__precio = precio
    self.calcularCalificacion(self.getCalificaciones())
    self.calcAudienciaEsperada(self.getCalificacion())
    self.checkEstadoCritico()
    Teatro.getInstancia().getObras().append(self)
    Obra.obras.append(self)

```

En ambos casos se usaba el self. para que no hubiera ambigüedad a la hora de apuntar los atributos en cada constructor. El (self) se usaba para añadir el objeto creado en cada constructor, pertinentemente, a su respectiva lista perteneciente.

– Implementación de un caso de enumeración

```

from enum import Enum

class Aptitud(Enum):
    CANTO = "CANTO"
    BAILE = "BAILE"
    DISCURSO = "DISCURSO"
    EMOCIONALIDAD = "EMOCIONALIDAD"
    IMPROVISACION = "IMPROVISACION"

```

```
class Actor(Artista):
    TASA = 1_000_000
    __BAJA_CALIFICACION = 3

    def __init__(self, nombre: str, id: int, edad: int):
        super().__init__(nombre, id)

        self.__generos = []
        self.__notas = []
        self.__tiempoActuado = []
        self.__reevaluacion = False
        self.__precioContrato = None
        self.__sexo = None
        self.__edad = edad

frncscp, 3 weeks ago • Clase Actor

# segun las aptitudes que estén en el enum
self.__aptitudes = [aptitud for aptitud in Aptitud]

# inicializar calificaciones en 0
self.__calificacionesAptitudes = [[0.0] * len(self.__aptitudes)]

# lista de listas vacías para almacenar calificaciones por cada aptitud
self.__historialCalificaciones = [[] for _ in range(len(self.__aptitudes))]

# agregar a la meta-clase serializadora el objeto Actor en las listas actores y artistas
Teatro.getInstancia().getActores().append(self)
Teatro.getInstancia().getArtistas().append(self)

self.__cuenta = CuentaBancaria(id, 999999999999999999999999)
```

Se implementa la enumeración aptitud ya que se inicializan las calificaciones de los artistas en la lista aptitudes, según las aptitudes en Aptitud, en ese orden.

- Implementación de multi herencia (Artista.py):

```
class Artista(Persona, conHorario):

    #constructor que se puede llamar solo con nombre e id
    def __init__(self, nombre: str, id: int, calificacion: float = 0, promedio: float = 0, clase: Clase = None):
        self.__nombre = nombre
        self.__id = id
        self.__calificacion = calificacion
        self.__promedio = promedio
        self.__cuenta = CuentaBancaria(id, 9999999999999999999999999999999.9)
        self.__clase = clase

        self.__horario = []
        self.__calificaciones = []
        self.__calificacionesPublico = []
```

Descripción de las Funcionalidades.

Descripción de la funcionalidad 1: Gestión de Ventas

Esta funcionalidad maneja la compra de tiquetes, suscripciones y asientos del teatro, además implementa el primer contacto con el cliente, puesto que aquí se le asigna un ID nuevo para que este sea reconocido y así poder seguir reconociéndose por este ID.

1. Verificación del cliente

- La funcionalidad recorre varios estados, su primera tarea es asegurarse de que el cliente es nuevo o no, puesto que si es un cliente que no es nuevo, este ya tendrá un ID asociado el cual podrá utilizar, o por el contrario, si es nuevo, se deberá crear un nuevo ID
- En caso de que el ID exista, el programa busca en un ArrayList donde se encuentran todos los clientes creados, para cada objeto de este Array verifica que efectivamente el ID ingresado por el usuario hace parte de los ID existentes, lo que realmente está pasando es que itera sobre los clientes y busca sus atributos id con el método getId(), y una vez encuentre un emparejamiento devuelve true, de lo contrario devolverá false,
 - Si esta verificación es correcta, se imprime “iniciando sesión...” como comprobante de que el ID es correcto
 - Además, imprimirá la información básica de cada cliente, como la última compra que realizó, su id correspondiente y el tipo de suscripción que tiene,
- Si por el contrario el usuario desea crear un Id nuevo, se elegirá uno aleatoriamente usando el método randomId que verificará que no tome id existentes, luego se le asignará por defecto que no ha hecho ninguna compra y que su suscripción es básica

2. Adquirir Suscripción

- La primera interacción real con el cliente será preguntarle si desea mejorar su suscripción, de manera que si este es nuevo, al tener una suscripción básica por defecto existe la posibilidad de que el pueda mejorarla, o incluso si es un cliente antiguo pueda adquirir una mejor.
 - Se le mostrarán todas las opciones de suscripción que se tienen, estas se tomarán del enum de Suscripciones, cada uno de ellos tendrá un precio asociado y un asiento a desbloquear
 - Todas las interacciones como compras realmente serán tratadas como reservas, puesto que sólo se asignará realmente la suscripción a el cliente una vez la compra se haya realizado, es decir, una vez que la funcionalidad termine se validará que la compra se haya realizado

2. Escoger Obra

- Para escoger una obra lo que se buscará será iterar sobre la lista de todas las obras disponibles verificando que la fecha de inicio sea correcta, es decir que se puede comprar , evitando que se compre funciones ya acabadas.
 - Cada obra tiene asociado un nombre, un precio y una calificación la cual definirá el mismo precio, es por eso que para que el cliente sepa qué función está comprando se desplegará un menú con todas las obras disponibles para ese momento, junto con su precio y su duración
 - Pueden existir diferentes funciones que estén asociadas a la misma obra e incluso a la misma sala , lo que las diferenciaban sería la fecha, es por eso que pueden existir funciones aparentemente iguales pero no lo son, para esto se desplegará otro menú con todas las funciones asociadas a la obra seleccionada , para que según el horario de preferencia , el cliente pueda escoger.
- ### 3. Escoger Silla
- Algo tan sencillo como podría ser asignar una silla, en este programan el manejo de esta no lo es en absoluto, ya que al implementar una suscripción y diferentes tipos de sillas para cada suscripción en específico, teniendo en cuenta que no son excluyentes, es decir que una suscripción de nivel máxima puede acceder a cualquier asiento, mientras que una de nivel intermedio solo podrá acceder a las inmediatamente igual o inferiores
 - Se itera sobre el ArrayList que se obtiene de la sala asociada puesto que esta es la que define la cantidad de sillas, luego se obtiene el valor de la silla, es decir el tipo de la silla que está definida por la primera letra del código asociado, esta se compara con el tipo de suscripción del cliente, haciendo un tipo de resta entre tipos, si suscripción es mayor que silla es por que en efecto puede escogerla
 - Otro impedimento que tendría el cliente a la hora de escoger silla es que esta ya esté ocupada , esto se resuelve mediante un menú en el que las sillas ocupadas se presentarán vacías, dejando en claro que no pueden ser usadas, esto se hace mediante una verificación junto con la clase silla en la que se busca que si algún cliente ya tiene asociado esta silla en su atributo tiquete de la clase Tiquete y si si, imprimirá este espacio vacío, o por el contrario imprimirá el código correspondiente

3. Imprimir Factura

- Una vez se eligió la función deseada, el cálculo del precio se hace bajo una medición exhaustiva , se calcula cuantos tiquetes de esa función en específico se han vendido y además según la calificación de los clientes dependiendo de qué tan buena o mala sea, y por último el cliente tiene un objeto del enum Suscripción que nos servirá para saber a qué beneficios podría acceder, es decir que descuento debería ser aplicado.
- Por último en la impresión de factura hay un factor importante, ya que aquí se verán reflejados las compras que realizo, como por ejemplo la suscripción, y queremos que solo aparezca en la factura si es la primera vez que tiene esta suscripción ya que esto significa que la acaba de comprar.

- Por último se imprime el horario en el que se realizó la compra, junto con el valor ahorrado y con el mensaje de agradecimiento por comprar con nosotros.

. Interacción de componentes

1. Cliente y Tiquete

- **Interacción:** Un cliente compra uno o más tiquetes para asistir a una función. Cada tiquete está asociado a un cliente y contiene información como el asiento reservado y la función correspondiente.
 - El Cliente proporciona sus datos (ID, nombre, etc.) para asociarse al tiquete.
 - El Tiquete registra la compra del cliente, la función seleccionada y la silla asignada.

2. Cliente y Tesorería

- **Interacción:** Cuando un cliente realiza una compra (de tiquetes o suscripción), la tesorería registra la transacción.
 - El Cliente genera una solicitud de compra.
 - La Tesorería registra el monto pagado y lo asocia al cliente en cuestión, actualizando los ingresos totales del sistema.

3. Cliente y Silla

- **Interacción:** Un cliente selecciona una silla al momento de comprar un tiquete.
 - El Cliente elige la silla deseada para una función específica.
 - La Silla verifica si está disponible y se asigna al cliente una vez confirmada la compra.

4. Cliente y Suscripción

- **Interacción:** Un cliente puede adquirir una suscripción para recibir beneficios adicionales.
 - El Cliente compra suscripción que le otorga descuentos y sillas exclusivas.
 - La Suscripción asocia beneficios al cliente.

5. Cliente y Función

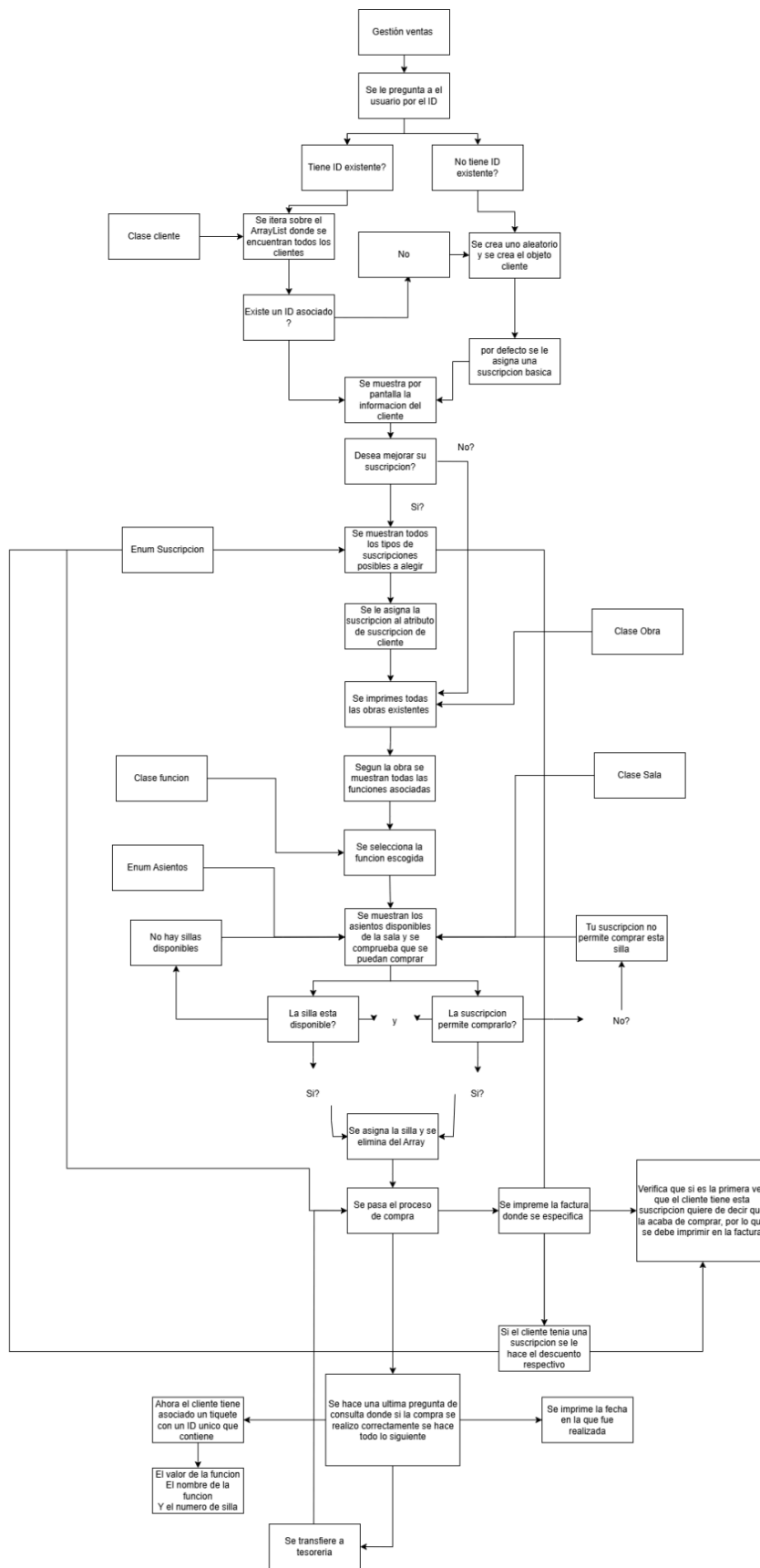
- **Interacción:** Un cliente compra tiquetes para asistir a una función específica.

- El Cliente selecciona una función disponible según su interés.
- La Función valida la disponibilidad de sillas y actualiza el registro de asistentes.

6. Tiquete y Tesorería

- **Interacción:** Cada vez que se compra un tiquete, la transacción se registra en la tesorería.
 - El Tiquete genera un costo total basado en la función, silla y posibles descuentos.
 - La Tesorería registra el monto de la compra

Diagrama de interacción.



Descripción de la funcionalidad 2: Gestión de Empleados

La funcionalidad de Gestión de Empleados es una pieza clave del sistema, diseñada para administrar de manera eficiente los recursos humanos del teatro. Esta funcionalidad abarca desde la contratación hasta la evaluación del desempeño de los empleados, asegurando un funcionamiento fluido de las operaciones. Se enfoca en la gestión de deudas, la asignación de trabajos y la ejecución de pagos, considerando las necesidades del teatro y las capacidades de su personal.

1. Gestión de deudas y contratos:

- Este proceso comienza evaluando si hay deudas por pagar a los diferentes empleados que tiene el Teatro (Seguridad, Aseador, Profesor). El sistema utiliza un ciclo for para ir iterando uno por uno de los empleados dentro de la lista donde se encuentra, y ejecutando el pago en caso de que se le deba plata al empleado
- Luego de realizar este proceso, empieza a mostrar la lista de empleados por categoría, al finalizar este proceso, le da la posibilidad al usuario de contratar o despedir empleados, donde la contratación se da por tipo de empleado, las cuales se muestran una lista de candidatos y se elige aquel que el usuario quiera. Por otro lado, se realizan directamente utilizando el ID del empleado, el cual puede extraerse de las listas previamente impresas. Al ser despedidos, los empleados reciben el pago correspondiente por el trabajo realizado, más un 20% adicional como liquidación. Si el teatro tiene deudas pendientes con ellos, estas se suman al pago final.

2. asignación y verificación de trabajos:

- Este módulo comienza analizando la cantidad de funciones programadas, diferenciando entre los empleados de Seguridad y los de Aseo. Evalúa cuántas tareas le corresponden a cada empleado, ajustando los procesos de forma independiente para cada tipo de personal. Esto significa que la cantidad de trabajos asignados a los empleados de Seguridad no es la misma que para los de Aseo, ya que cada grupo se gestiona según sus propias características y responsabilidades.
- Una vez calculada la cantidad de trabajos correspondientes, se inicia el proceso de asignación de horarios, comenzando con la primera función del día. Para los empleados de Seguridad, su horario coincide exactamente con el de la función, mientras que, para los empleados de Aseo, su horario comienza al finalizar la función y termina de limpiar 15 minutos después. Posteriormente, el sistema evalúa los horarios de las funciones restantes, asegurándose de que no haya solapamientos con los horarios ya asignados. Este proceso continúa hasta completar la cantidad de trabajos programados para cada empleado. Al finalizar la asignación inicial, queda una lista de trabajos disponibles, que corresponde a las tareas restantes tras la división equitativa del trabajo. Estas tareas se asignan de una en una a los empleados, verificando que no se solapen con los horarios previamente definidos. En caso de que no se cumpla esta condición, la función no podrá ser vigilada o aseada, dependiendo del tipo de empleado en el que ocurrió el solapamiento.

- Al finalizar el proceso de asignación, comienza la evaluación del trabajo, que se basa en analizar la experiencia del empleado para determinar la calidad de su desempeño. Si al trabajador se le asignaron tareas de alta dificultad, existe la posibilidad de que haya cometido errores, especialmente si su experiencia es limitada. Por otro lado, si el empleado cuenta con suficiente experiencia y las tareas asignadas no son complejas, es probable que las haya completado correctamente y sin inconvenientes.

3. Ejecución de pagos y evaluación de desempeño

- Este proceso se enfoca principalmente en los aspectos financieros del teatro, evaluando las diferentes condiciones en las que este puede encontrarse. Durante la evaluación, se verifica si tanto el teatro como los empleados alcanzaron las metas previamente establecidas. Si el teatro cumple su objetivo, se otorga a todos los empleados una bonificación del 30% de su sueldo. Además, aquellos empleados que lograron sus metas individuales reciben un 15% adicional de bonificación, reconociendo su desempeño y contribución al éxito general además de pagarles sus respectivas deudas.
- Si el teatro no cuenta con fondos suficientes para cubrir el total de los pagos, se procede a ejecutar el pago de la mitad de la suma del sueldo y la deuda correspondiente a cada empleado. La otra mitad se asigna como una nueva deuda que el teatro deberá saldar en el futuro. Este proceso asegura que los empleados reciban al menos una parte de su pago, mientras que el teatro gestiona su situación financiera de manera equilibrada.
- Un proceso que se realiza sin la necesidad de efectuar los pagos es la evaluación del desempeño de los empleados, que se centra en analizar el cumplimiento de las metas personales de cada uno. Aquellos empleados cuya meta sea menor a 0 serán despedidos, ya que no cumplen con el nivel de desempeño requerido por el teatro. A estos empleados se les ejecutará el pago correspondiente, incluyendo su liquidación. Tras este proceso, el sistema organiza la lista de empleados en función de su desempeño, desde aquellos con las metas más altas hasta los de menor rendimiento.

Interacción entre componentes:

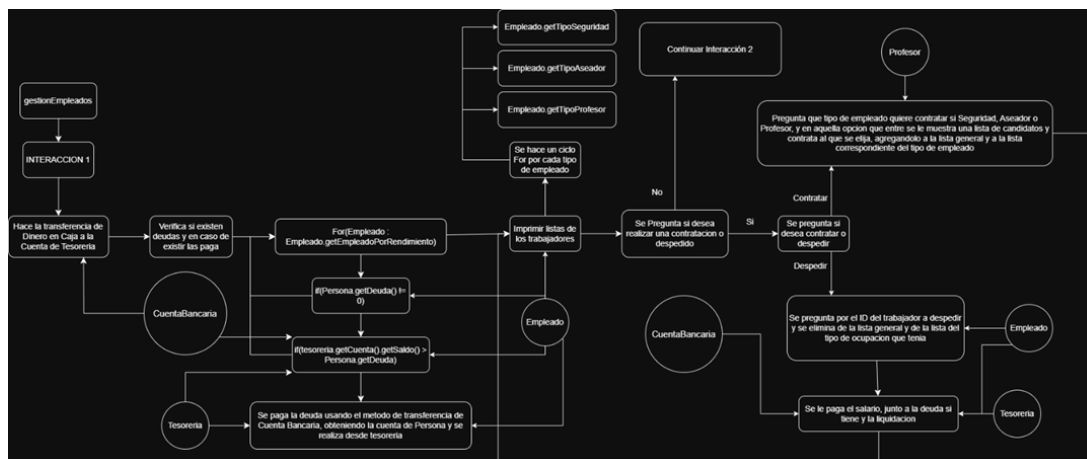
1. **Tesorería - Empleado:** La clase Tesorería interactúa con Empleado para administrar pagos, verificar deudas y realizar transferencias de fondos. Tesorería utiliza los métodos de Empleado para acceder a datos como salario, deudas pendientes y meta semanal.
2. **Tesorería - CuentaBancaria:** Tesorería utiliza la clase CuentaBancaria para realizar transacciones financieras. Este componente facilita las transferencias entre las cuentas bancarias de la tesorería y los empleados.
3. **Empleado - CuentaBancaria:** Cada empleado tiene asociada una cuenta bancaria. Las transacciones de pago, como la liquidación del salario y el manejo de deudas, se gestionan a través de esta interacción.

4. **Empleado - Sala:** Los empleados, en especial los de tipo "Aseador" y "Seguridad", interactúan con las Salas. Los empleados de seguridad supervisan funciones asignadas en las salas, mientras que los aseadores limpian las salas después de las funciones.
5. **Empleado - Función:** Los empleados de tipo "Seguridad" y "Aseador" están asignados a funciones específicas. Seguridad administra horarios para garantizar la protección de cada función, y los aseadores limpian según el horario de las funciones.
6. **Sala - Función:** Las salas contienen funciones que tienen horarios específicos. Cada función utiliza la sala como su espacio principal, y su estado (como limpieza) está vinculado a las interacciones con los empleados asignados.

En este sistema, cada clase interactúa con múltiples clases de manera simultánea, lo que crea un modelo integrado y complementario. Esta interdependencia entre las clases asegura que el sistema funcione como un todo coordinado, donde cada componente contribuye al funcionamiento eficiente del resto.

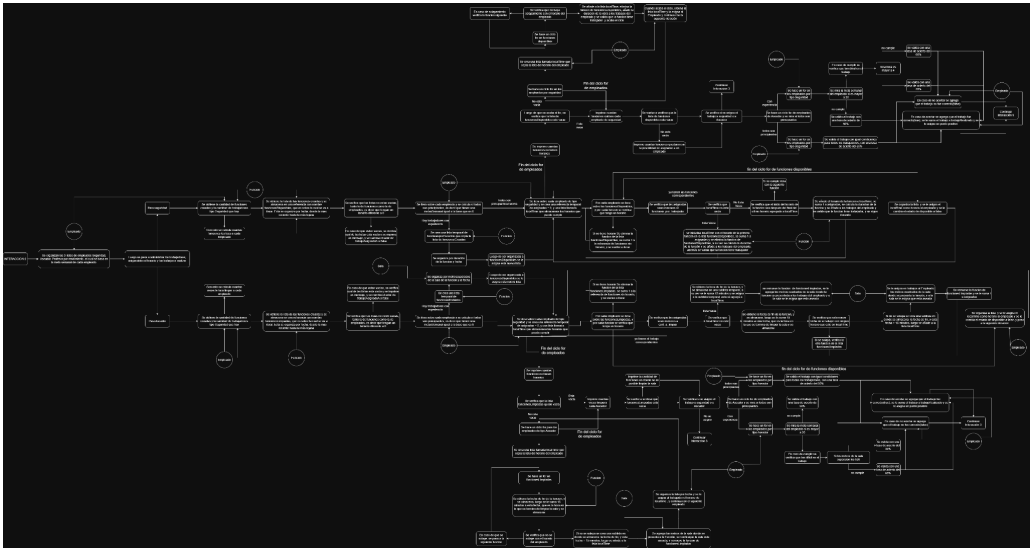
Diagramas de interacción:

Interacción 1:



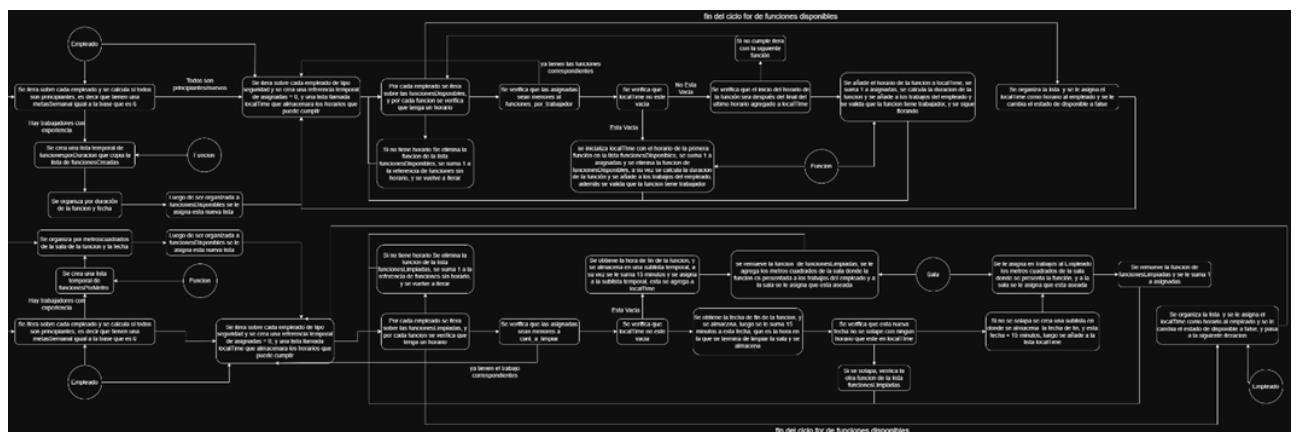
Interacción 2

General:

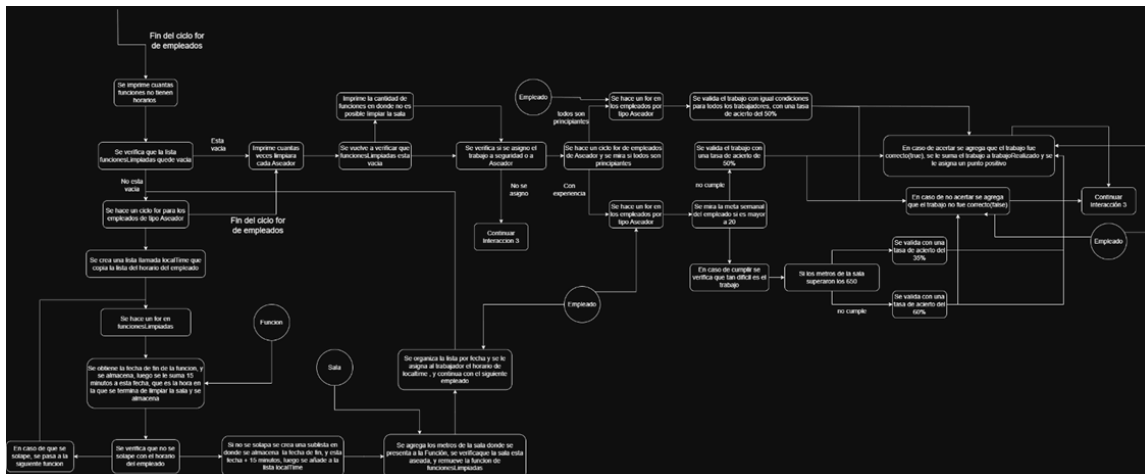


```

graph TD
    Interaccion2[INTERACCIÓN 2] --> Organiza[Se organiza las 3 listas de empleados Seguridad, Asesor, Profesor por rendimiento, el cual se basa en la media sumatoria de cada empleado]
    Organiza --> Largo[Largo se pasa a administrar los trabajadores, empezando el horario y los trabajos a realizar]
    Largo --> ParaSeguridad[Para Seguridad]
    Largo --> ParaAsesor[Para Asesor]
    Largo --> ParaProfesor[Para Profesor]
    
    ParaSeguridad --> ObtSeg[Se obtiene la cantidad de funciones creadas y la cantidad de trabajadores tipo Seguridad que hay]
    ObtSeg --> CalcSeg[Con esto se calcula cuántas funciones le toca ir a cada Empleado]
    CalcSeg --> ObtSeg
    
    ParaSeguridad --> FuncSeg((Funcion))
    FuncSeg --> ObtSeg
    
    ObtSeg --> ListaSeg[Se obtiene la lista de las funciones creadas y se almacena en una referencia con nombre funcionesChaponeses, que es sobre la cual se va a buscar. Ésta se organiza por fecha, desde la más reciente hasta la más lejana]
    ListaSeg --> VerSeg[Se verifica que las listas no estén vacías, tanto la de funciones como la de empleados, es decir que tengan un tamaño diferente a 0]
    VerSeg --> ImpSeg[En caso de que estén vacías, se verifica cual de las listas está vacía y se imprime un mensaje, y se cambia el valor de trabajadoresSeguridad a false]
    ImpSeg --> Sala((Sala))
    
    ParaAsesor --> ObtAsesor[Se obtiene la cantidad de funciones creadas y la cantidad de trabajadores tipo Seguridad que hay]
    ObtAsesor --> CalcAsesor[Con esto se calcula cuántas funciones le toca ir a cada empleado]
    CalcAsesor --> ObtAsesor
    
    ParaAsesor --> FuncAsesor((Funcion))
    FuncAsesor --> ObtAsesor
    
    ObtAsesor --> ListaAsesor[Se obtiene la lista de las funciones creadas y se almacena en una referencia con nombre funcionesAsesor, que es sobre la cual se va a buscar. Ésta se organiza por fecha, desde la más reciente hasta la más lejana]
    ListaAsesor --> VerAsesor[Se verifica que las listas no estén vacías, tanto la de funciones como la de empleados, es decir que tengan un tamaño diferente a 0]
    VerAsesor --> Sala
    
    ParaProfesor --> ObtProf[Se obtiene la cantidad de funciones creadas y la cantidad de trabajadores tipo Seguridad que hay]
    ObtProf --> CalcProf[Con esto se calcula cuántas funciones le toca ir a cada empleado]
    CalcProf --> ObtProf
    
    ParaProfesor --> FuncProf((Funcion))
    FuncProf --> ObtProf
    
    ObtProf --> ListaProf[Se obtiene la lista de las funciones creadas y se almacena en una referencia con nombre funcionesProfesor, que es sobre la cual se va a buscar. Ésta se organiza por fecha, desde la más reciente hasta la más lejana]
    ListaProf --> VerProf[Se verifica que las listas no estén vacías, tanto la de funciones como la de empleados, es decir que tengan un tamaño diferente a 0]
    VerProf --> Sala
  
```

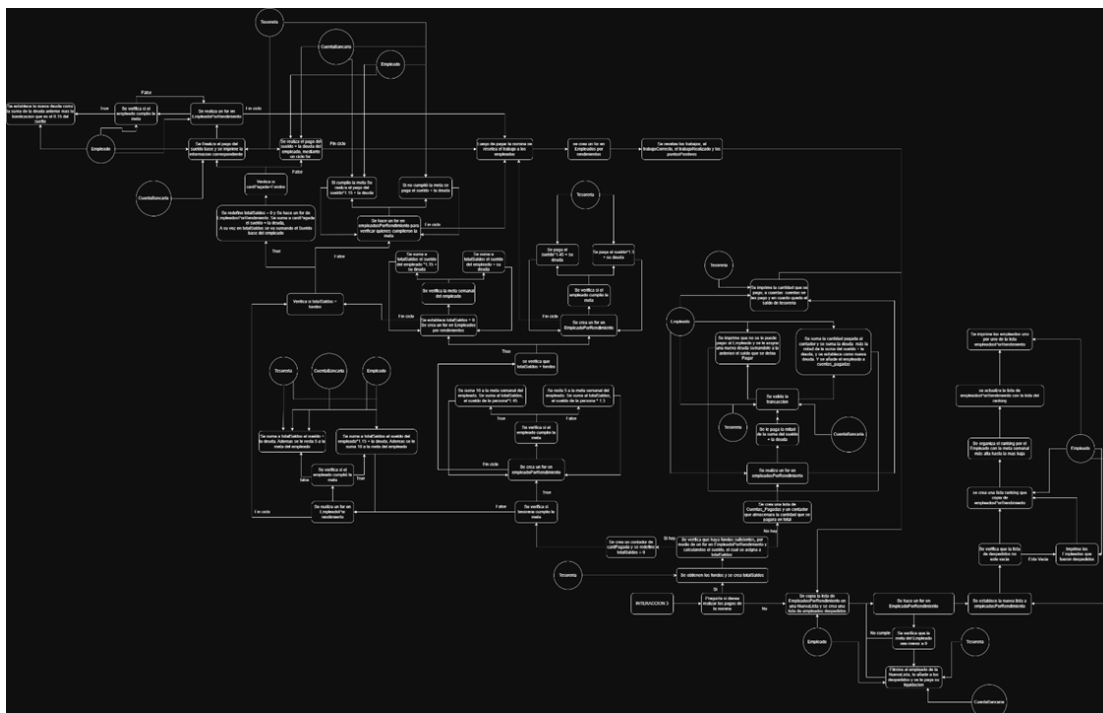
[illegible]

Parte 3: Cuerpo-Abajo-Aseador

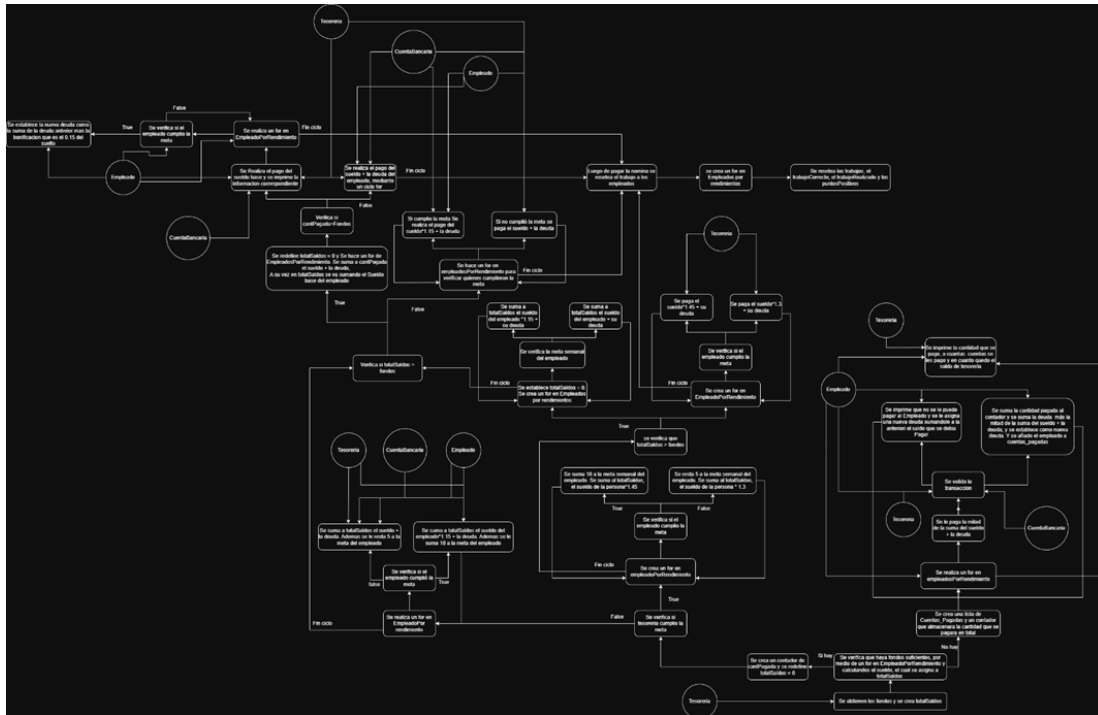


Interacción 3

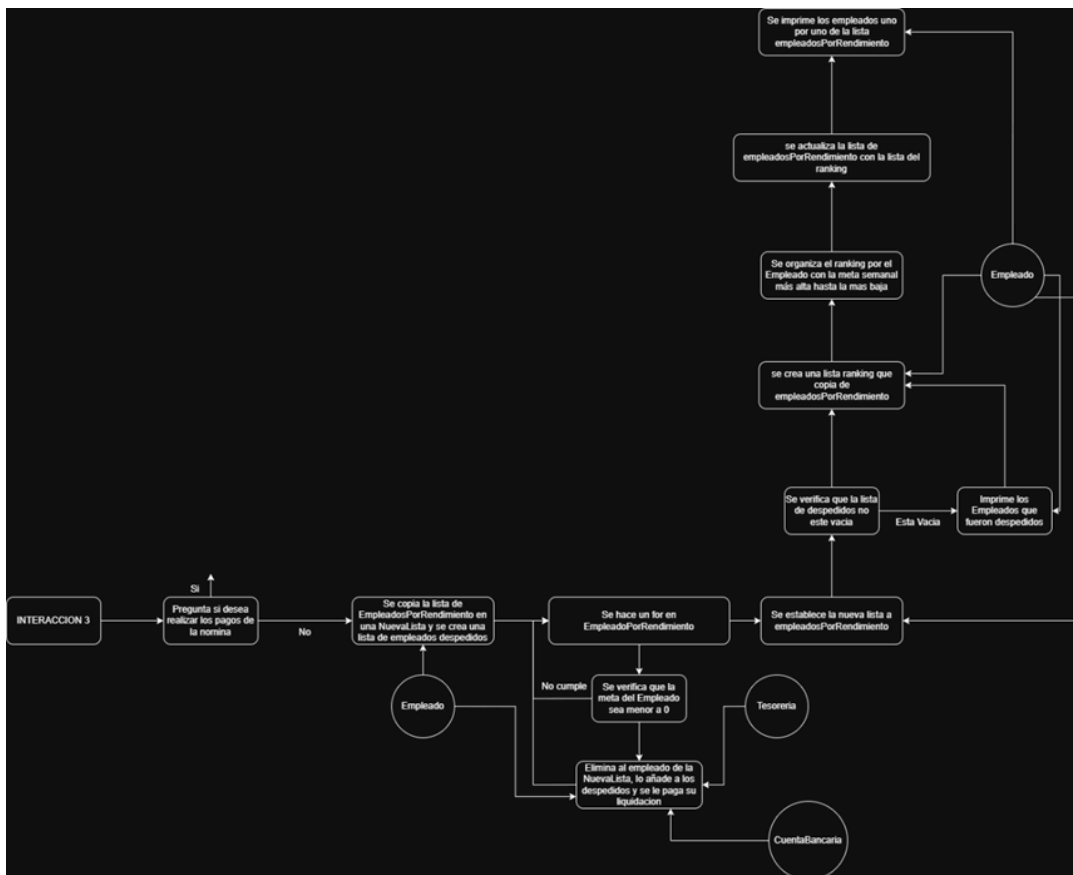
General:



Parte 1: Lógica pago nomina



Parte 2: cuerpo



DESCRIPCIÓN DE LA FUNCIONALIDAD 3. GESTIÓN DE OBRAS

Esta funcionalidad implementa los procesos necesarios para acompañar al usuario en la gestión y manejo de las obras del teatro escuela Carlos Mayolo, la funcionalidad cuenta con un camino que te lleva desde la creación de las obras, con todos sus elementos, te ayuda a crear funciones en base a las necesidades de la obra y al terminar te genera un apoyo visual que te permite reconocer y asociar el trabajo que acabas de hacer creando funciones en un formato tipo horario.

1. Creación de obras para el uso de franjas horarias.

El primer proceso que se lleva a cabo es la creación de obras con funciones estelares predefinidas para poder inicializar las franjas horarias que usamos para definir en qué momentos asignar las obras nuevas de algún género en específico. Estas obras solo se crean para inicializar las franjas y no pueden crearse funciones de estas.

2. Mostrar menú de obras.

Este es el primer paso que incluye una respuesta dentro de consola, para enseñar este menú debe iterar sobre la lista estática obras ya serializada e imprimir todos los objetos que no tengan como nombre NOTFORITE (obras solo para implementación como se definió arriba), además imprime con ellas un índice que las identifica en la lógica del programa.

3. Elección de una obra.

En este punto el usuario debe ingresar el índice que acompaña a la obra, el programa cuenta con varios controles para asegurarse que soporte errores en caso de que el usuario envíe un tipo de dato incorrecto o un entero fuera del rango de los índices de obras ya creadas. En este punto el usuario puede decidir crear un nuevo objeto obra usando la última opción del menú.

4. Creación de una nueva obra (si aplica)

En caso tal de que el resultado del paso anterior concluya en que el usuario desea crear una nueva obra, el programa le solicitará los datos que necesita para poder crear el objeto, almacenarlo y usarlo en un futuro para la creación de funciones. Los datos que se solicitan durante este proceso son:

- Nombre de la obra
- Lista de actores y los roles específicos que interpretarán (para esto se accede a la lista de actores que se modifica desde la funcionalidad Gestión de clases y se pide al usuario elegirlos uno por uno, una obra puede igualmente no tener reparto)
- Género de la obra (Este viene de una enumeración, por tanto, son opciones definidas)
- Director (Este tiene que ser especialista en el género previamente ingresado, en caso tal que no se encuentren directores asociados a ese género o se desee ingresar uno nuevo se ofrece la opción de crear nuevo director)
- Costo de producción
- Duración de la obra en un formato HHMMSS, si se ingresan menos de 6 dígitos el programa hace el arreglo para que no se afecten los cálculos

5. Definir el número de funciones a crear para la obra seleccionada

Con base a la obra seleccionada, se calculan cuántas funciones se recomiendan crear, con este número en cuenta recibe del usuario la cantidad de funciones que se desean generar, si el desvío de la cantidad de funciones con respecto a las recomendaciones es mayor a 2 se genera una alerta (Alerta de demasiadas funciones si lo excede en más de 2 y de pocas funciones si tiene 2 funciones menos que las recomendadas), luego de

generar la alerta el usuario puede decidir continuar igualmente o volver y definir una cantidad distinta de funciones.

6. Creación de las funciones

Este paso involucra varios pasos que ocurren a nivel interno de la lógica, por lo que al usuario solo se le presenta el resultado final, al ya haber definido la cantidad de funciones que se le darán a la obra, se itera dentro de la lista de salas para comprobar su disponibilidad en la franja horaria también ya definida, además, se verifica que los actores también se encuentren disponibles y empieza a asignar las funciones; el sistema prioriza que las funciones se reproduzcan a lo largo de la semana, además, una función solo puede iniciar o en hora en punto o a las y 30, sin importar su duración o la de la obra previa a ella.

Al terminar de crear cada función se imprime un mensaje que verifica su correcta creación, incluyendo su horario y la sala asignada.

7. Imprimir horarios por sala

Una vez se llega a este punto se accede a la lista funcionesCreadas que se encuentra en la clase facilitadora de uso de datos Teatro, con esta información se imprimen los horarios de cada sala para los próximos 7 días, mostrando en cada sala las funciones a llevar a cabo, con el nombre de la obra y la hora, el horario imprime los siete días de la semana así no haya funciones programadas para algún día en específico y se asegura de ordenar las funciones de la manera en la que se presentan.

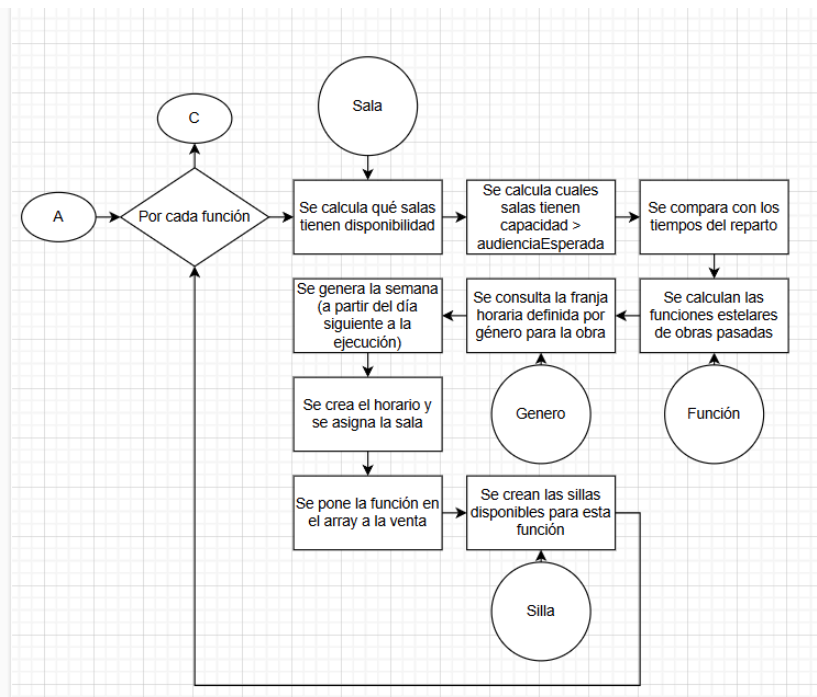
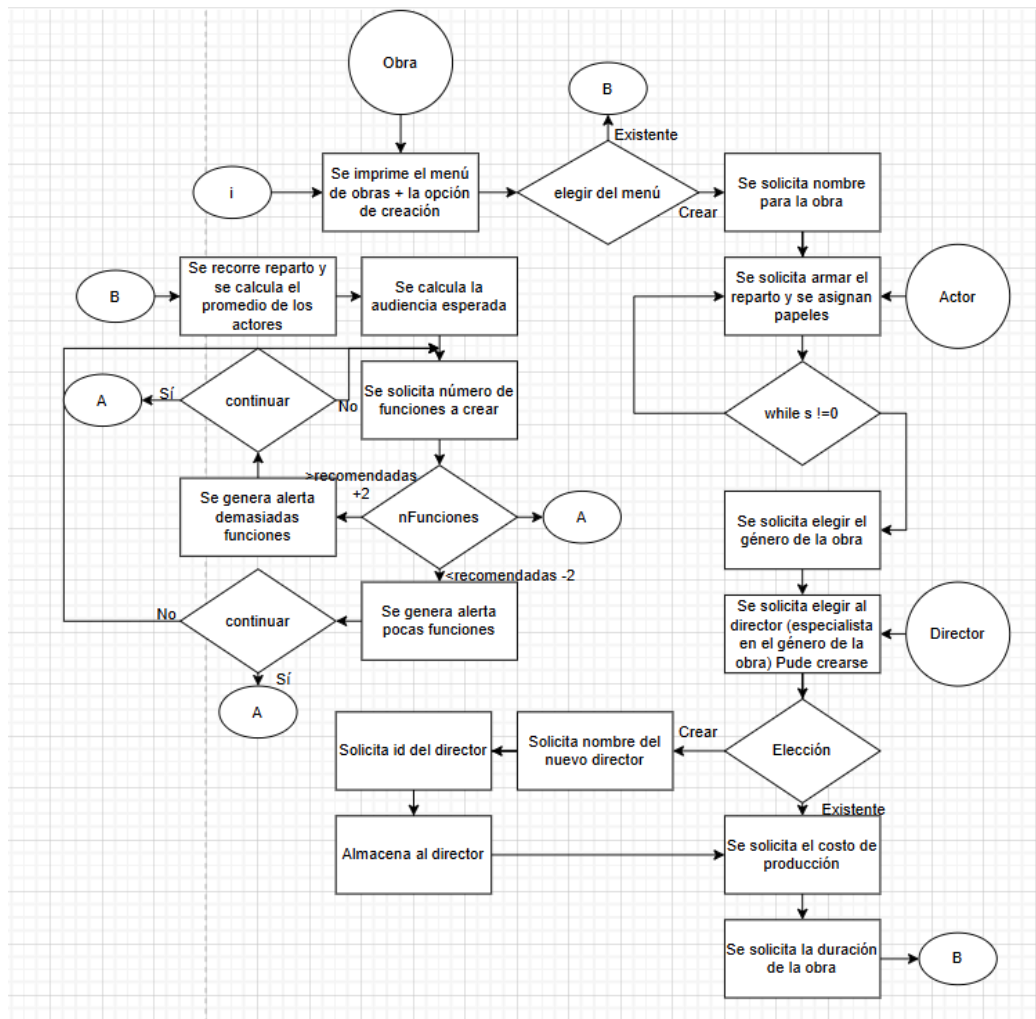
8. Listar clientes interesados.

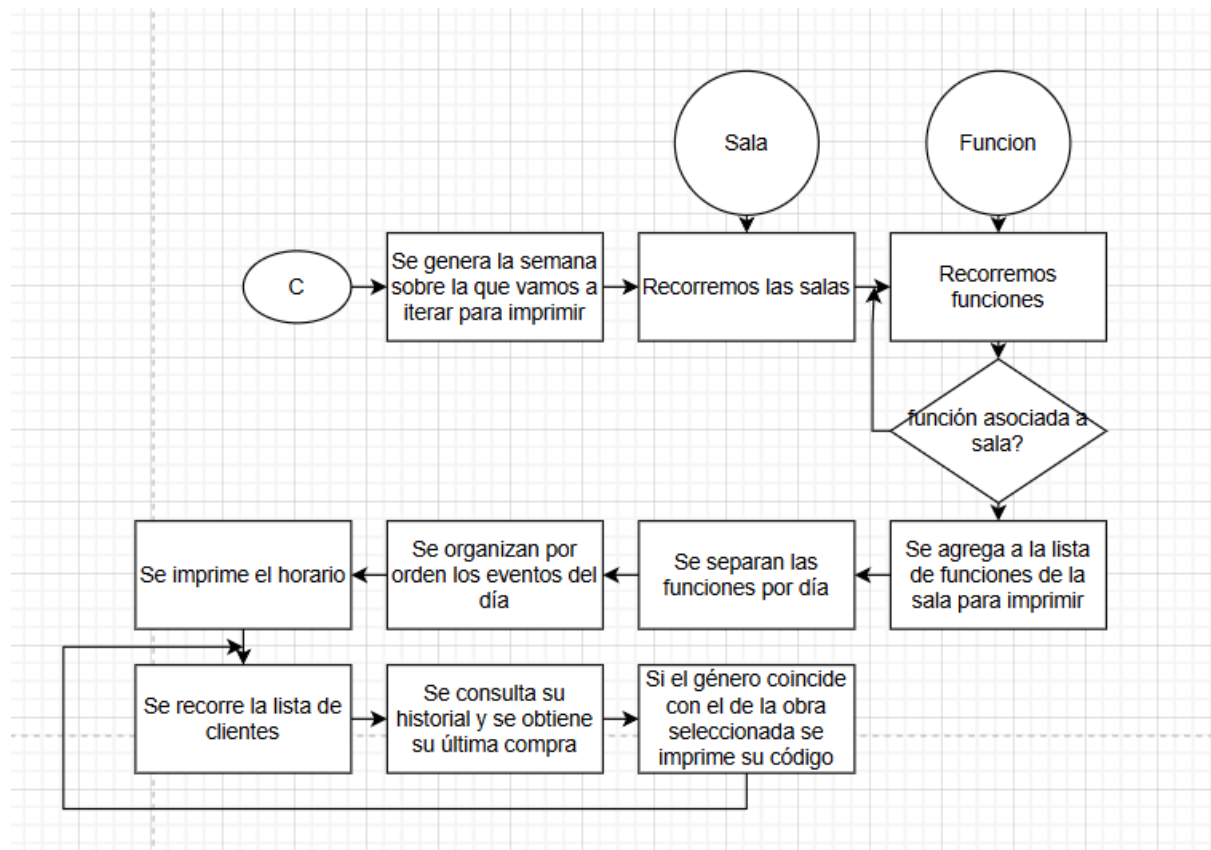
En este último paso, se itera sobre los clientes ya creados, en caso tal de que existan, y se imprimen como lista los códigos de aquellos cuyo género favorito sea el mismo de la función que acabamos de programar, con esto podemos conocer a quienes podría interesar la función.

INTERACCIÓN ENTRE COMPONENTES

- 1. Obra:** Durante la creación, se asignan actores, un director, género, duración y costo. Además, durante la programación se generan funciones asociadas a ella.
 - 2. Actor:** Se asigna al elenco de una obra durante su creación, su disponibilidad puede influir en la programación de funciones.
 - 3. Director:** Se selecciona al momento de configurar una obra.
 - 4. Sala:** Su disponibilidad y capacidad son cruciales al momento de crear funciones. Además, sirve como contenedor para luego imprimir el horario.
 - 5. Función:** Se genera en la segunda interacción del programa. Además, se incluye en la programación para que puedan empezar a venderse tiquetes.
 - 6. Cliente:** Su interés o géneros favoritos deben verificarse para imprimir la lista de potenciales clientes.
-

DIAGRAMAS DE INTERACCIÓN





Descripción de la funcionalidad 4: Gestión de Clases

La funcionalidad de gestión de clases es una pieza clave del sistema, diseñada para planificar y gestionar las clases de capacitación en un teatro-escuela. Esta funcionalidad se centra en los actores como principales beneficiarios, permitiendo la creación de horarios, la asignación de salas y profesores, la facturación de las clases y la evaluación del progreso de los actores. A continuación, se describen en detalle las interacciones y los componentes involucrados.

1. Identificación del Artista:

- El proceso comienza preguntando al usuario por el ID del artista. El sistema utiliza el método estático de la clase Artista “buscarArtistaPorId(id)” para buscar al artista en la base de datos.
- Si el artista no existe, se ofrece la posibilidad de crear un nuevo artista, especificando su nombre y tipo (actor o director).

- En el caso de los directores, se informa al usuario que no toman clases.
- Si se crea un actor, se inicializan sus calificaciones mediante el método casting() de la clase Empleado.

2. Consulta de calificaciones:

- Una vez identificado el artista:
 - Si es un actor, se muestran sus calificaciones actuales separadas en dos categorías:
 - **Calificaciones de calificadores:** Evaluaciones dadas por profesores, a través del método “getCalificacionesAptitudes()”.
 - **Calificaciones del público:** Opiniones dadas por el público, a través del método getCalificacionesPublico()”.
 - Si el actor es nuevo, se inicializan sus calificaciones con ayuda de un profesor disponible.

3. Análisis de obras críticas:

- En esta etapa, se presentan las obras en “estado crítico” (aquellas con calificaciones muy bajas en ciertos aspectos), a través del método estático de la clase Obra “mostrarObrasCriticas()”.
- Se revisan las aptitudes críticas de cada obra y las calificaciones de los actores responsables de dichas aptitudes. Si hay deficiencias, se notifica al actor para tomar acción.

4. Programación de clases:

- Si el usuario decide programar una clase para un actor:
 - Se recomienda un área de mejora basada en las peores calificaciones del (actor.getCalificacionPorAptitud()).
 - Se determina el nivel de la clase:
 - **Introducción:** Para calificaciones menores a 3.0.
 - **Profundización:** Para calificaciones entre 3.0 y 4.0.
 - **Perfeccionamiento:** Para calificaciones mayores a 4.0.
 - Se asigna una sala disponible que cumpla con las condiciones de horario y limpieza(Sala.isDisponible()).
 - Se asigna un profesor capacitado(Profesor.tieneEspecializacion()) y disponible para la clase.
 - Se calcula el costo de la clase según su nivel:
 - Introducción: \$50,000.
 - Profundización: \$75,000.
 - Perfeccionamiento: \$90,000.
 - El actor realiza el pago a través de su cuenta bancaria, transfiriendo los fondos a la tesorería del teatro.

5. Evaluación y retroalimentación:

- Tras completar la clase, el profesor asignado evalúa al actor, generando una nueva calificación para el área trabajada.
 - Si la calificación obtenida es perfecta (5.0):
 - Se reembolsa el costo de la clase al actor como incentivo.
 - Si no hay mejora en el área seleccionada tras una clase (Actor.huboMejora()):
 - Se agenda una nueva clase en el mismo ámbito.
 - Si no hay mejora después de cuatro intentos (Actor.noHaMejoradoEnCuatroIntentos()):
 - El nivel del área se reduce para ajustar las expectativas de aprendizaje.
 - Los profesores reciben puntos positivos como recompensa:
 - 1 punto por clases de Introducción.
 - 2 puntos por clases de Profundización.
 - 3 puntos por clases de Perfeccionamiento.
-

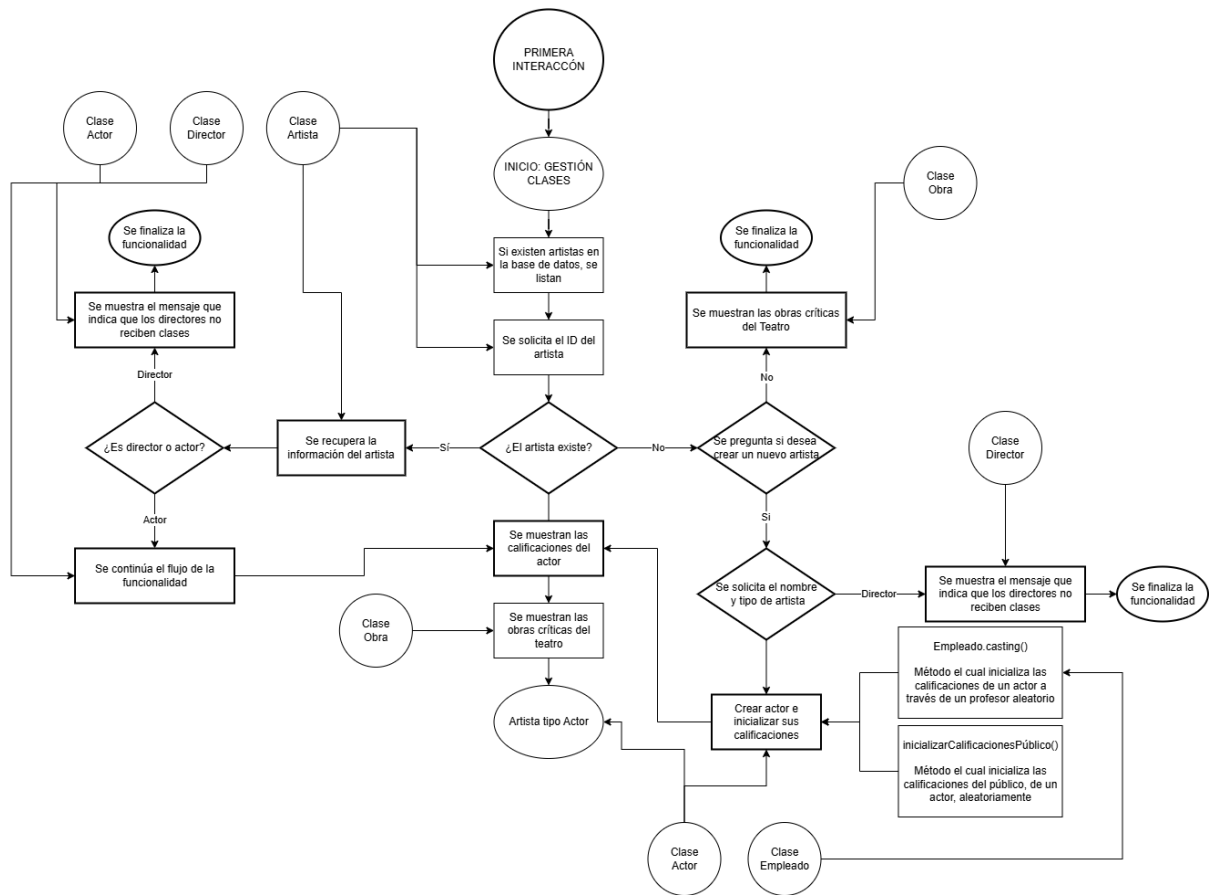
Interacción entre componentes:

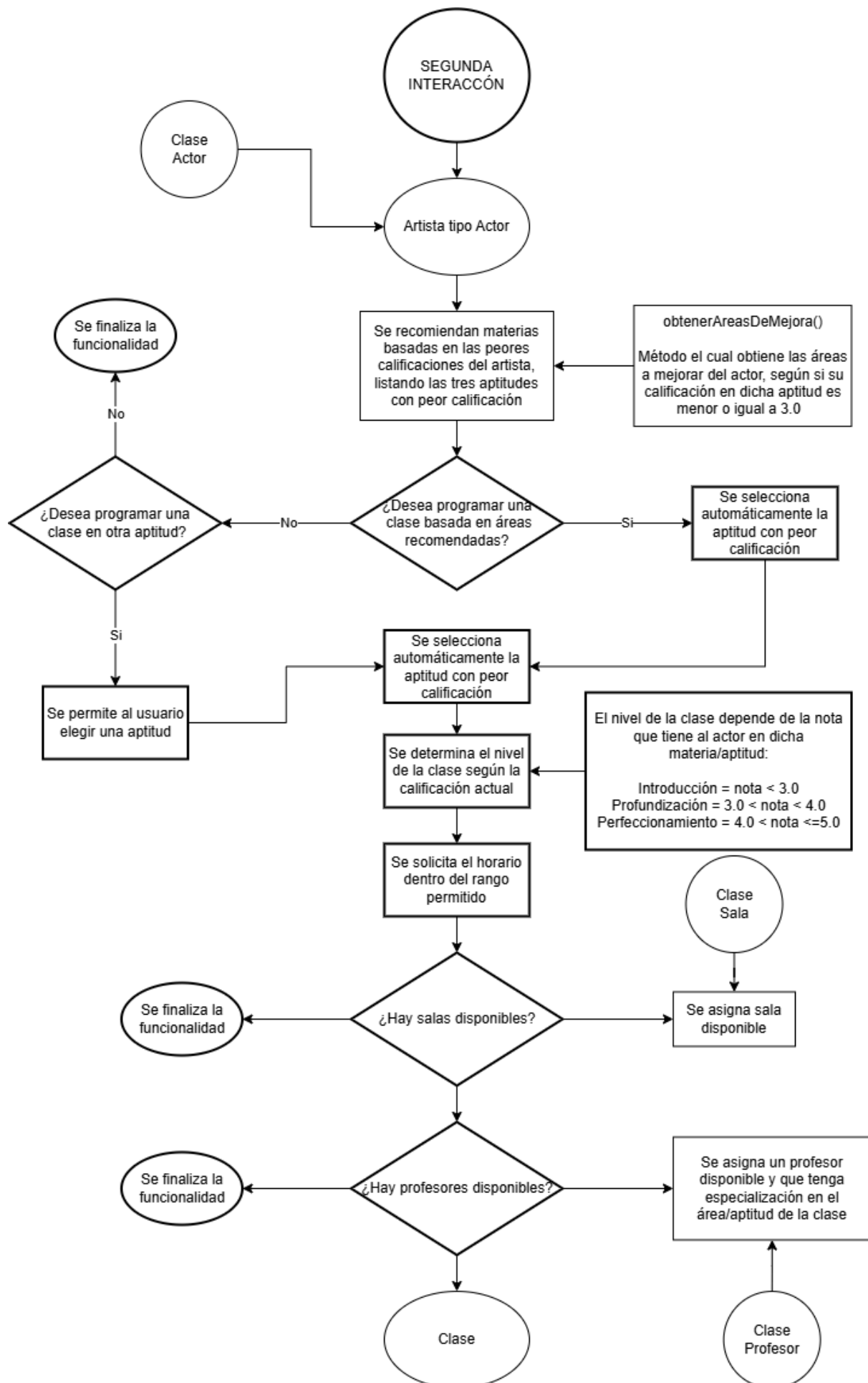
1. **Artista:** Artista actúa como una clase base que representa a los artistas. La clase Actor hereda métodos específicos como la gestión de calificaciones y el historial de mejora. Los directores no reciben clases, pero son gestionados dentro del sistema como parte de los artistas.
2. **Actor:** Subclase de Artista. Incluye métodos para gestionar aptitudes, calificaciones y áreas de mejora.
3. **Sala:** Las salas juegan un papel crítico para asegurar que haya un espacio disponible y en condiciones adecuadas para las clases. El sistema consulta su disponibilidad y estado de limpieza.
4. **Profesor:** Empleado.casting() inicializa las estadísticas de los actores nuevos, asignando calificaciones iniciales. Los profesores son seleccionados en función de su especialización y disponibilidad, asegurando que las clases sean impartidas por profesionales capacitados.
5. **Tesorería:** Administra los pagos realizados por los actores y registra el ingreso de fondos.
6. **CuentaBancaria:** Facilita las transacciones entre el actor y la tesorería.
7. **Obra:** Identifica las funciones teatrales, incluyendo las áreas críticas que requieren atención. Las obras en estado crítico sirven como indicador para priorizar las clases de los actores, fomentando una mejora en las áreas de bajo desempeño.

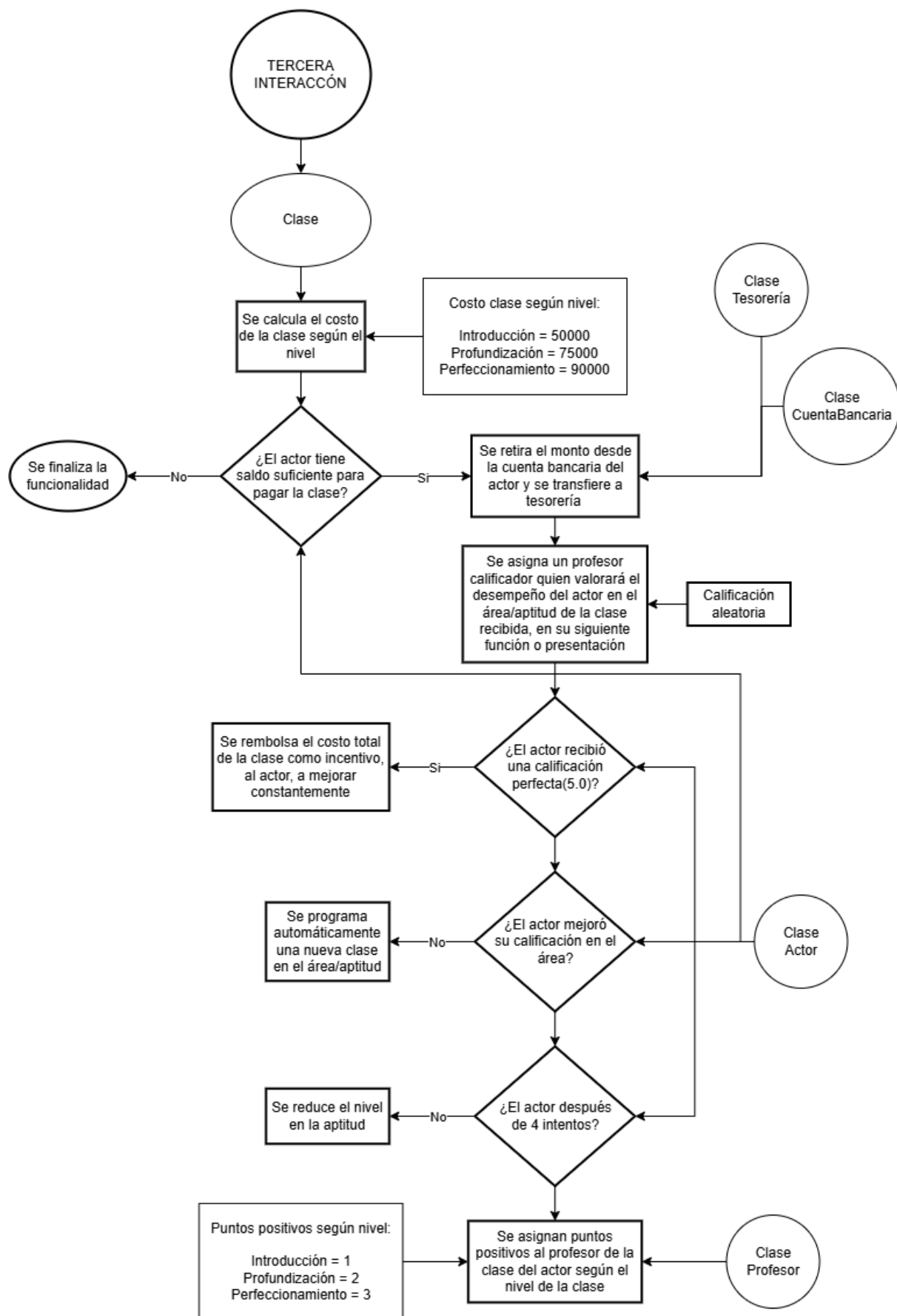
8. **Empleado:** Clase base para gestionar al personal del teatro, incluyendo a los profesores.

Esta colaboración entre clases permite que el sistema gestione eficientemente las clases, desde la identificación de necesidades hasta la evaluación del aprendizaje y la mejora continua del actor.

Diagrama de interacción de la funcionalidad 4: Gestión de Clases







Descripción de la funcionalidad 5: contratación de actores

La funcionalidad se encarga de la contratación de actores, a través de un filtrado en base a las características del objeto Actor, una vez se escoge el actor, se prosigue con la transacción del costo del contrato en la tesorería, por último, se le asigna al actor el nuevo horario definido en el contrato. En la contratación se involucran las clases Actor, Tesorería, Teatro, CuentaBancaria y Cliente.

1. Verificación de cliente y primera ronda de filtrado.

- Empieza requiriendo la creación o el inicio de sesión de un cliente tipo empresa, en cualquiera de los dos casos, se revisa si la entrada es válida y si el ID también
- Si eligió un cliente nuevo, se revisa en la lista con objetos tipo Cliente que es atributo de Teatro si el ID es existente, de ser así se levantará la excepción `errorIdExiste` con un mensaje avisando la invalidez de la entrada.
- Si eligió la opción de cliente existente, se corroborará en la lista de objetos de tipo Cliente si tal cliente existe, de no ser así, se levantará la excepción `errorIdNoExiste` con un mensaje avisando la invalidez de la entrada.
- El usuario inicia sesión con un cliente tipo Empresa, sea nuevo o existente (en caso de ser nuevo, está definido en el constructor que se agregue a la lista de objetos de tipo Cliente en la instancia de tipo Teatro)
- Una vez el cliente esté configurado, se harán preguntas para definir el tipo de actor que se busca con la contratación, esta etapa incluye tipo de rol en la obra, la evaluación de la calificación del género de obra elegido para cada actor, evaluación de las aptitudes y revisión de conflictos de horario.
- Para filtrar actores por el tipo de rol en la obra se evalúa la opción escogida, en caso de ser un rol principal, se validará que se remuevan aquellos cuya calificación sea menor a una constante `CALIFICACION_ALTA`. Si, por lo contrario, se requiere contratar para un rol secundario, se removerán aquellos actores con calificación por encima de la mencionada constante.
- Para seguir filtrando actores en función de su género y aptitudes, se escogerá un solo género y una sola aptitud, se leerán las calificaciones asociadas a las enumeraciones Género y Aptitud en el caso escogido, y en base a una constante se revisará si deben o no ser eliminados, similar al filtrado para tipo de rol.
- Una vez se tenga la lista filtrada con la primera ronda de preguntas, el programa preguntará por la fecha de contratación, mostrando los 7 días próximos desde la fecha

actual, luego, se pedirá hora de inicio y fin de la contratación, por último, se revisará que el horario cumpla con los siguientes lineamientos:

1. El horario debe ocurrir entre las 08:00 y las 22:00.
 2. El horario no debe durar más de 8 horas.
 3. El horario debe durar por lo menos 4 horas.
- Se evaluará por cada actor en la lista si su horario tiene espacio libre en el horario de contratación, de no ser así, será removido de la lista

2. Pre-clasificación de actores y búsqueda avanzada

- En esta etapa del código se evalúa si algún actor está en reevaluación, de ser así será removido de la lista porque no se considera apto para contratar.
- Si el cliente tipo Empresa ya existía, se creará una lista auxiliar, y se iterará en la lista de Actores para revisar si habían sido contratados por el cliente.
- De ser así, la lista de actores será reordenada, donde aquellos actores que aparezcan en el historial de la empresa tengan prioridad sobre los que no.
- Se preguntará si se desea hacer búsqueda avanzada, que tendrá más características del actor en cuenta. De no ser así, la lista de actores habrá terminado de filtrarse. Si se hace búsqueda avanzada, se creará una lista de tuplas de la siguiente manera:

$$[(actor_1, cuenta_1), \dots, (actor_n, cuenta_n)]$$

Donde se instancia una lista compuesta de tuplas, de forma que se puedan agregar objetos de tipo Actor y números enteros por cada índice.

- Se pregunta por el intervalo de edad que se busca que el actor pertenezca, están definidos los siguientes:
 1. Infantil (menores de 15 años)
 2. Juvenil (de edad mayor o igual a 15 años y menores de 25)
 3. Adulto (de edad mayor o igual a 25 y menor a 70)
 4. Adulto mayor (de edad mayor o igual a 70)

Por cada actor, se debe evaluar su edad y removerse si no pertenece al intervalo de edad escogido. Una vez se tenga la lista actualizada, se iterará en la lista de contadores, y si el objeto Actor sigue apareciendo en la lista, a su número contador se le sumará 1.

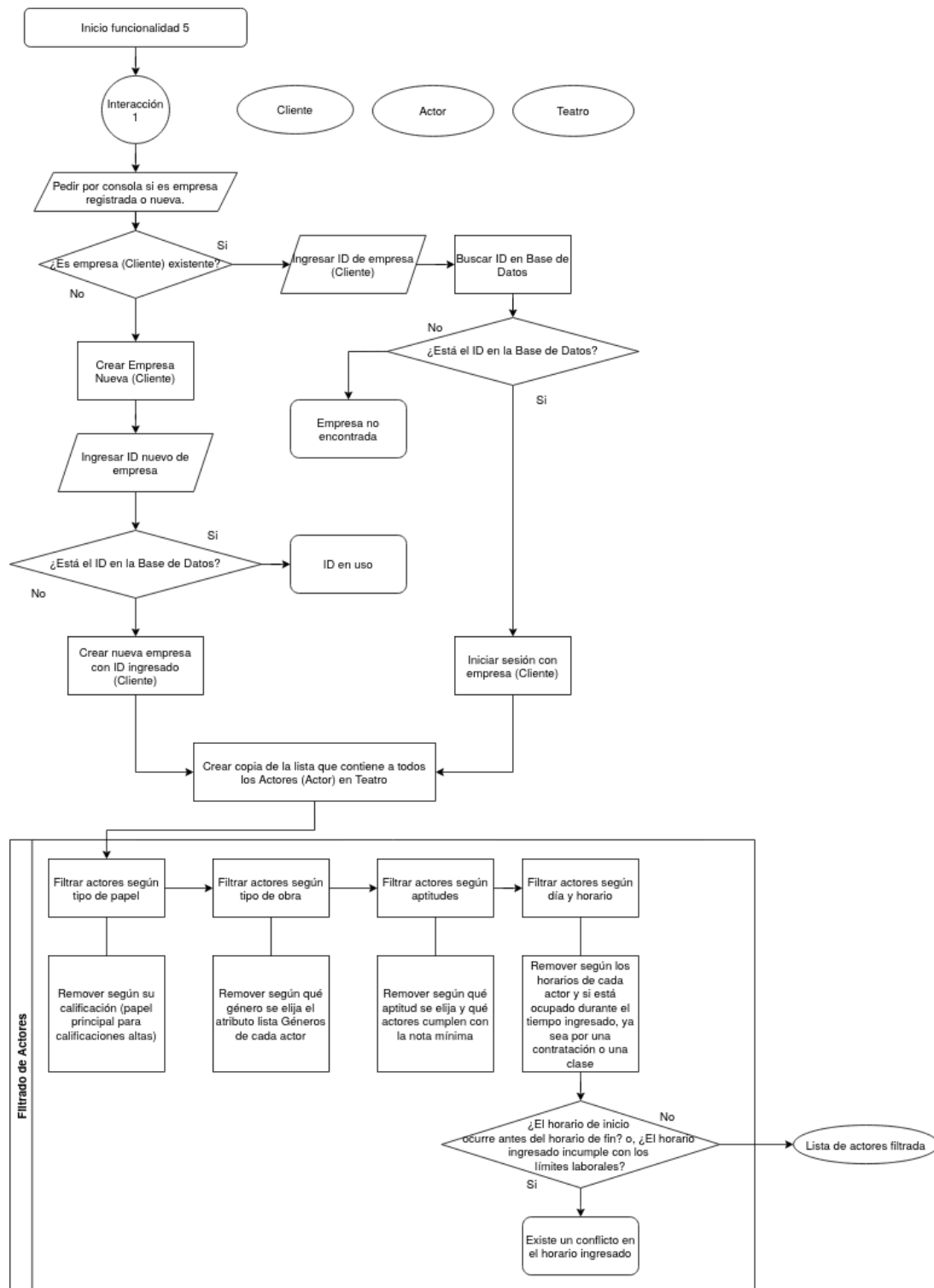
- Se pregunta por el sexo que debe tener el actor, y se hace un proceso similar a la pregunta de intervalo por edad, escogiendo entre masculino y femenino. Análogamente, se sumará en el número contador si el objeto actor sobrevive al filtrado.
- Acaba el filtrado por búsqueda avanzada, y se revisa si existen actores que cumplan con al menos una característica avanzada, de ser así, la funcionalidad puede seguir, y mostrará la cantidad de actores encontrados. De lo contrario, se retornará un mensaje diciendo que no se encontraron actores con las características especificadas.

3. Contratación de actor y asignación de horario

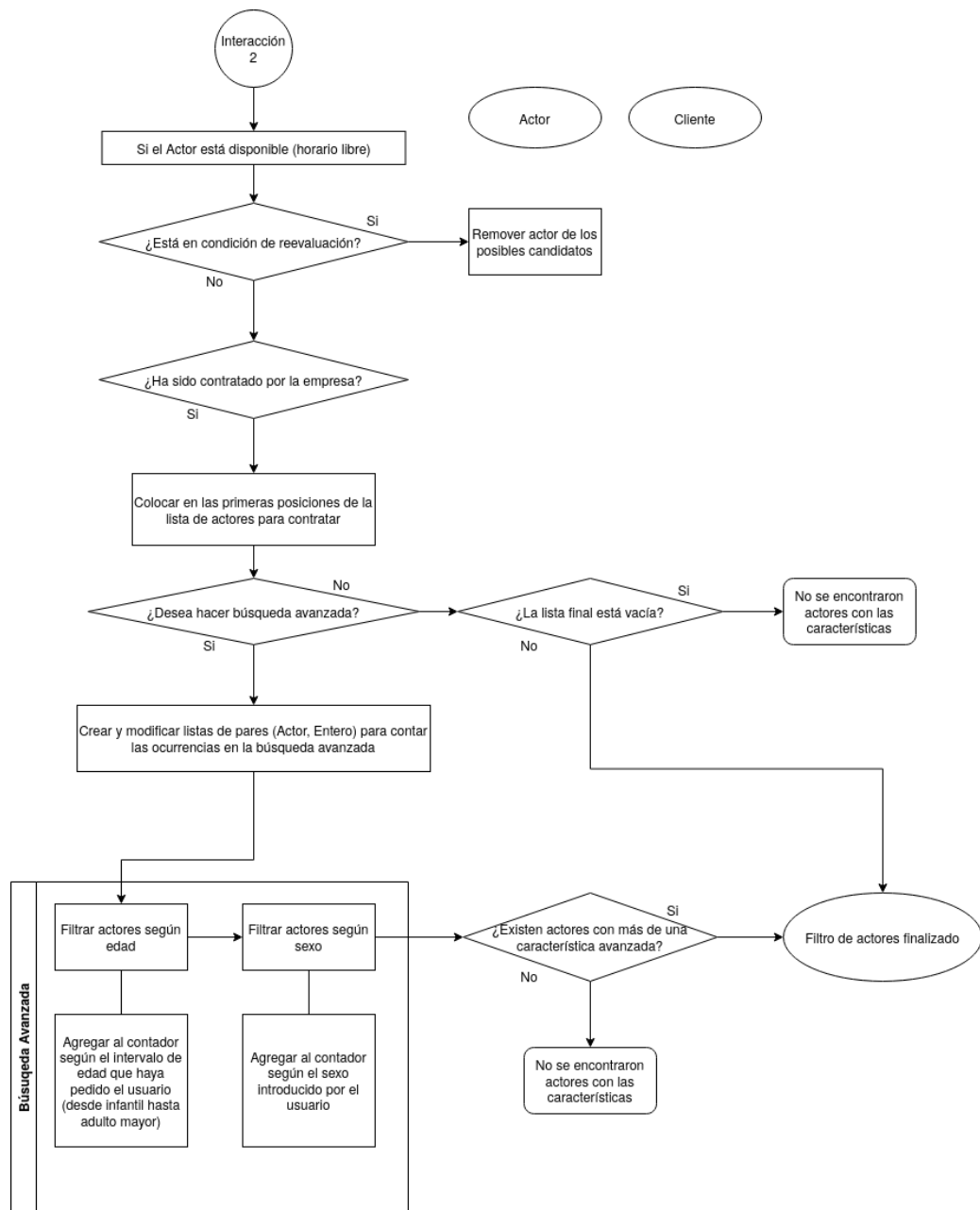
- Si se llega a este punto de la funcionalidad y existen actores para contratar, se pedirá el presupuesto de contratación, mostrando el rango de valores que cobran los actores para la duración especificada en el horario. Si el presupuesto es menor al valor mínimo de contratación, se retornará un mensaje diciendo que no hay actores para el presupuesto.
- Se desplegará un cuadro de lista con los posibles actores a contratar, una vez se elija uno, se abrirá una ventana de diálogo de confirmación donde se mostrará su nombre, precio de contratación, edad y calificación.
- Si se procede con la contratación, desde el objeto Cliente tipo Empresa se llamará el método pagarContratoActor, que agrega el actor al historial de la empresa y realiza la transferencia al objeto Tesorería, a través de su objeto CuentaBancaria

Diagramas de interacción de la funcionalidad 5: contratación de actores

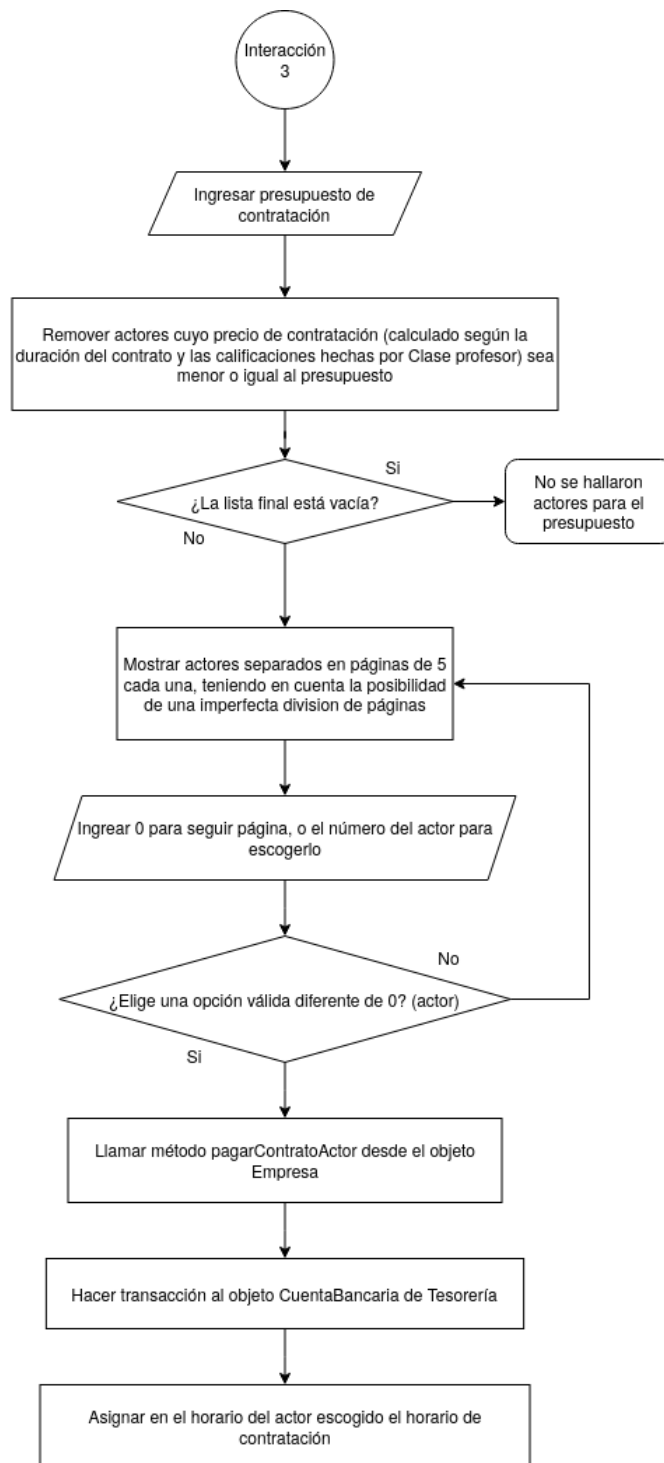
Interacción 1



Interacción 2



Interacción 3



Manejo de excepciones

Excepciones:

Se crearon 11 excepciones de la cual, la padre es errorAplicacion y de esta derivan 2 grupos el errorEntrada y errorExistencia, además de errorSuscripcion que es una excepción aparte, los 2 grupos consisten en las siguientes excepciones:

errorEntrada:

- errorEntradaNoNumerica
- errorEntradaNula
- errorFormatoHorario

errorExistencia:

- errorIdExiste
- errorIdNoExiste

Descripción:

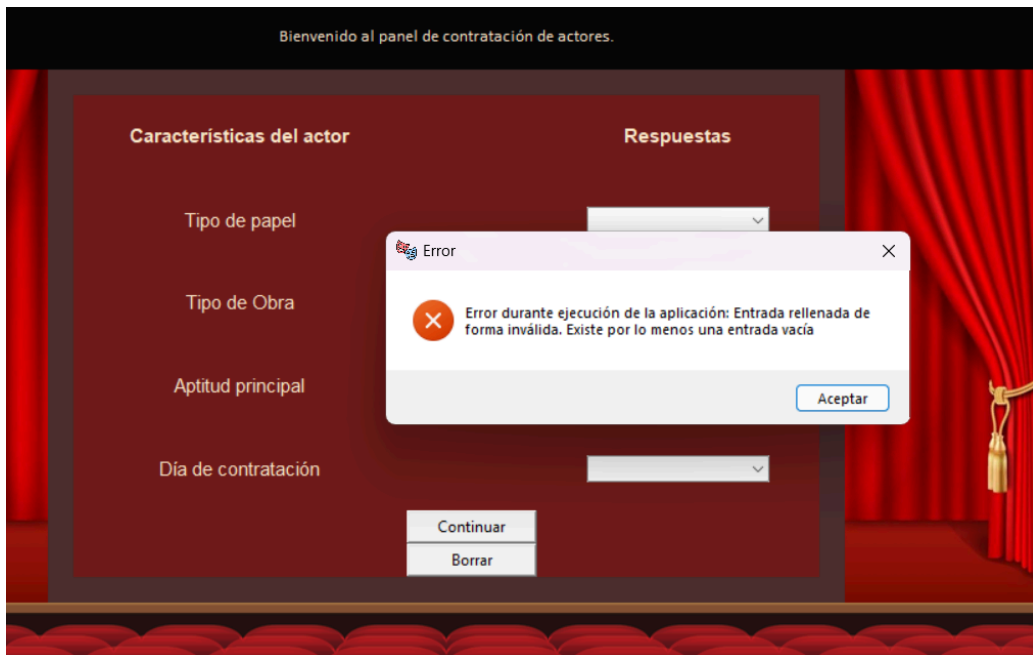
- errorEntrada: esta excepción se utiliza al momento en que el usuario digite una entrada incorrecta a la pedida en la sección correspondiente, el programa lance una alerta y pueda seguir ejecutándose el código, por tanto se utilizan diferentes excepciones para cada tipo de entrada pedida:
 - errorEntradaNoNumerica: ésta se ejecuta en caso de que el usuario no digite alguna entrada numérica
 - errorEntradaNula: Se ejecuta en caso de que no se haya digitado una entrada, es decir que este vacía
 - errorFormatoHorario: esta se ejecuta en caso de que el usuario no digite un horario en el formato correspondiente
- errorExistencia: esta excepción se utiliza al momento en que se verifica la existencia de algún objeto con la característica correspondiente, y así se pueda lanzar una alerta que le indique al usuario la existencia de este, por tanto se utilizan diferentes excepciones para cada tipo de objeto que el usuario puede digitar o pedir:
 - errorIdExiste: esta excepción se usa en caso de que el usuario haya creado un Id que ya se encuentra en la base de datos
 - errorIdNoExiste: esta excepción se usa en caso de que el usuario quiera acceder a un Id el cual no se encuentra en la base de datos
- errorSuscripcion: esta excepción se utiliza en el momento en que el usuario quiere comprar una silla que no pertenece a las disponibles para su tipo de suscripción

Uso de las excepciones en el código y en ejecucion:

errorEntrada:

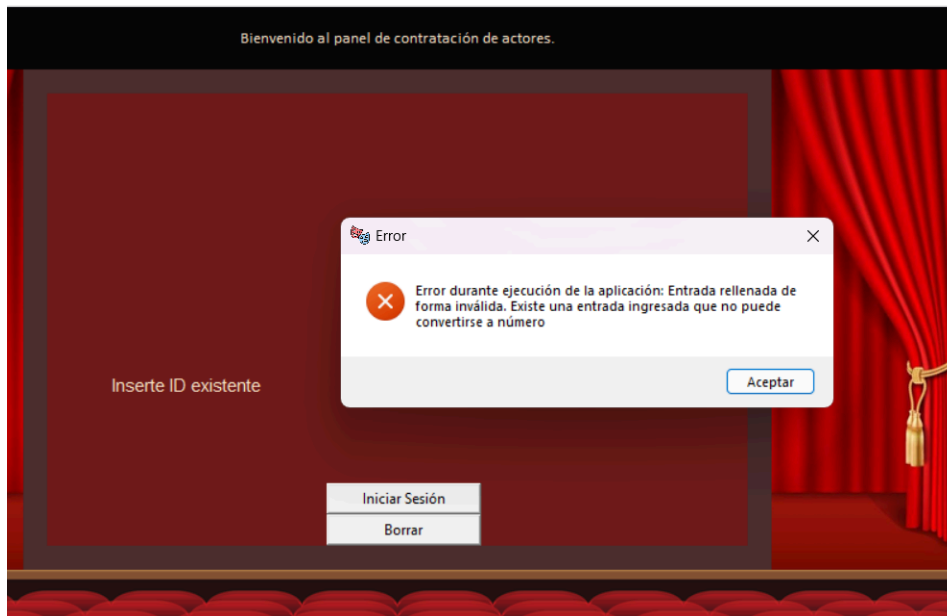
- errorEntradaNula

```
def validar_id(fieldframe : FieldFrame):
    try:
        fieldframe.gatherEntries()
        suscripcion = fieldframe.getValue("ID")
        validar(suscripcion)
    except errorEntradaNula:
        messagebox.showerror("Error", errorEntradaNula())
    return
```



- errorEntradaNoNumerica

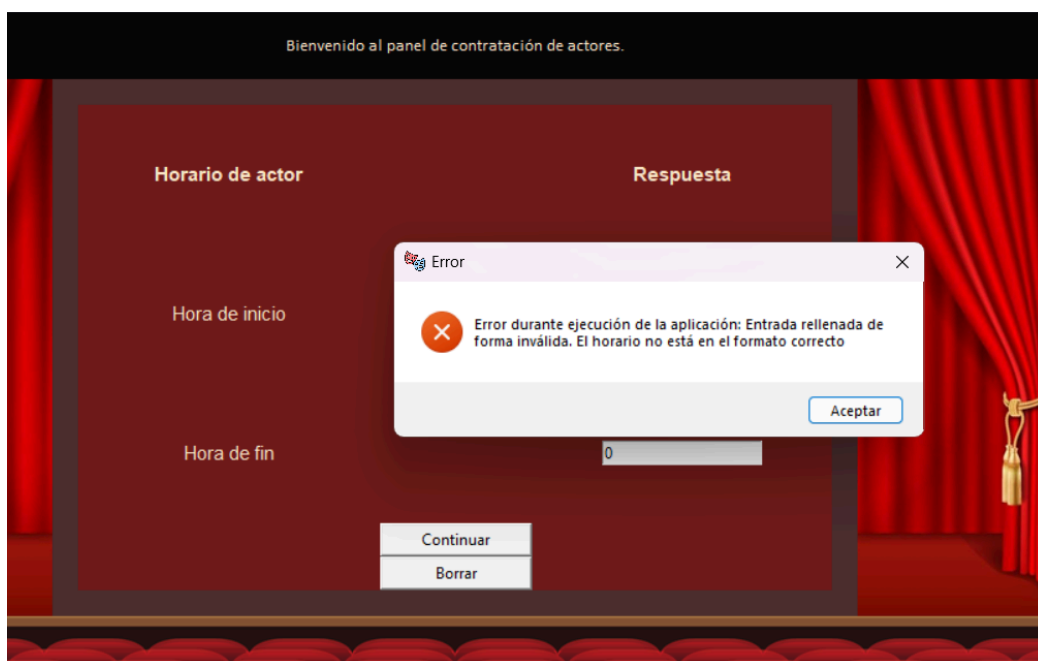
```
try:
    presupuesto = parseNumber(fieldframe, "Presupuesto", float)
except errorEntradaNoNumerica:
    messagebox.showerror("Error", errorEntradaNoNumerica())
    return
except errorEntradaNula:
    messagebox.showerror("Error", errorEntradaNula())
    return
```



- errorFormatoHorario

```
horaInicio = fieldframe.values[1].get()
horaFin = fieldframe.values[2].get()

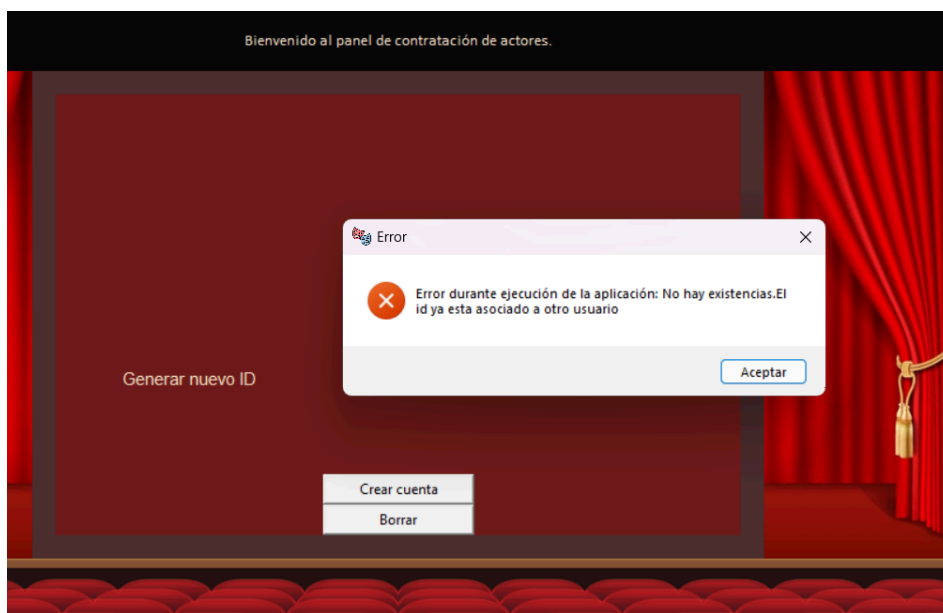
try:
    fecha, horaInicio, horaFin = Main.validar_formatos(fecha, horaInicio, horaFin)
except errorFormatoHorario:
    frncscp, 2 days ago * remove: errorHorario e implementaci
    messagebox.showerror("Error", errorFormatoHorario())
    return
```



errorExistencia:

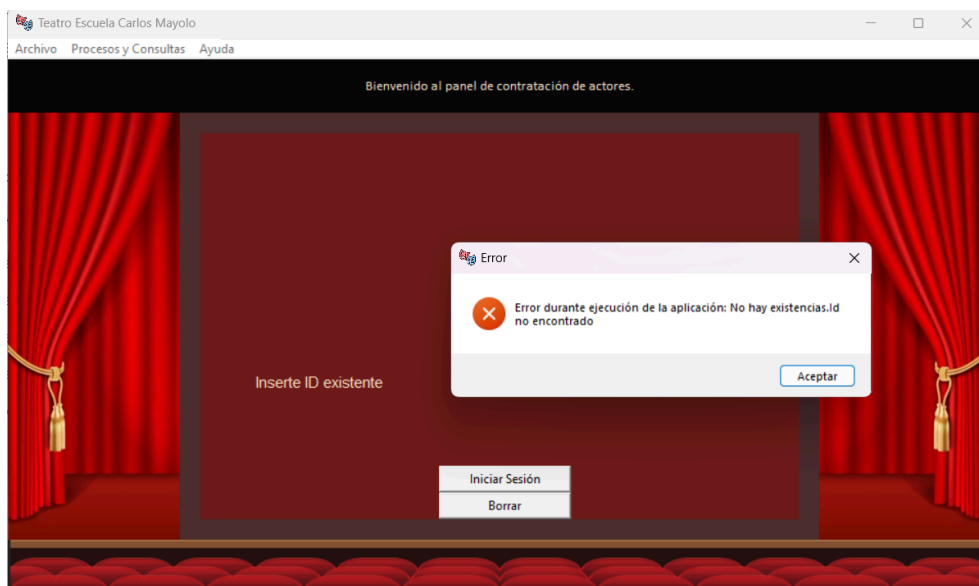
- `errorIdExiste`

```
def auth(fieldframe: FieldFrame, clientExists: bool) -> None:
    if clientExists:
        try:
            locateId(fieldframe)
        except errorIdNoExiste:
            messagebox.showerror("Error", errorIdNoExiste())
    else:
        try:
            createId(fieldframe)
        except errorIdExiste:
            messagebox.showerror("Error", errorIdExiste())
```



- `errorIdNoExiste`

```
def auth(fieldframe: FieldFrame, clientExists: bool) -> None:
    if clientExists:
        try:
            locateId(fieldframe)
        except errorIdNoExiste:
            messagebox.showerror("Error", errorIdNoExiste())
    else:
        try:
            createId(fieldframe)
        except errorIdExiste:
            messagebox.showerror("Error", errorIdExiste())
```



errorSuscripcion:

```
try :
    cliente.verificarSuscripcion(l[0])
    for i in funcion_elegida.getSillas():
        if i.getCodigo()==numero:
            funcion_elegida.getSillas()[indi].setCodigo("ocupado")

        indi += 1
    imprimir_factura(numero)

except errorSuscripcion:
    messagebox.showerror("Error", errorSuscripcion(1))
```

FieldFrame

La clase FieldFrame hereda de Frame (Tkinter), y es utilizada para generar formularios. Debido a que el programa depende de las entradas del usuario para muchas funcionalidades, es necesario estandarizar el proceso de creación de formularios adaptables, pues evita la reimplementación innecesaria de código y lo vuelve modular. Por defecto, si se le pasa una lista de criterios, genera un formulario genérico con valores de texto ya definidos, pero los elementos del formulario son editables: los títulos, botones, valores por defecto y habilitación de edición de valor también pueden ser escogidos al inicializar un objeto de esta clase.

La clase está pensada para ser adaptable a la cantidad de datos a pedir al usuario, por lo que las entradas de criterios (donde se le explica al usuario el dato a ingresar) y valores (entradas predeterminadas) se deben pasar al constructor como listas, lo que permite manejar varios formularios en uno, utilizar los botones Aceptar/Borrar para todos los valores y en general facilitar la implementación para múltiples valores. El FieldFrame está distribuido en tres columnas, de izquierda a derecha: en la primera, van todos los criterios junto a su título; en la segunda, van los botones de guardar y borrar; por último, en la tercera, van las entradas junto a su título.

También está incluida la posibilidad de usar cuadros de lista (combobox), en el caso que se quieran mostrar una cantidad de opciones finitas a elegir (por ejemplo, enumeraciones). De esta forma, FieldFrame está diseñada con el propósito de ser altamente abstracta y poder ser adaptada para cada funcionalidad según sea necesario.

Para instanciar un objeto de la clase, existen los siguientes argumentos:

- **root (Tk | Frame):** es la ventana donde se va a colocar el FieldFrame, puede ser un objeto Tk o un Frame.
- **títuloCriterios (cadena):** define el título que va a aparecer en la columna de criterios, su valor predeterminado es “Requerimientos”.

- **criterios (lista con cadenas):** almacena los criterios a pedir al usuario, aparecerán en la columna de criterios organizados de forma vertical.
- **tituloValores (cadena):** define el título que va a aparecer en la columna de valores (entradas), su valor predeterminado es "Por favor digite:".
- **valores (lista de cadenas | lista de listas de cadenas):** almacena los valores predeterminados para mostrar en las entradas, en el caso de usar cuadros de lista, por cada cuadro debe haber una lista de cadenas que almacene las opciones a elegir. Su valor predeterminado es None. En el caso que tenga el valor predeterminado, todas las entradas estarán vacías.
- **habilitado (lista):** almacena aquellos criterios cuyos valores pueden ser editables, su valor predeterminado es None. En el caso que tenga el valor predeterminado, todos los criterios serán editables.
- **combobox (booleano):** se define si el FieldFrame usará entradas de texto para el usuario o cuadros de lista. Su valor predeterminado es False (en este caso, se usarán entradas de texto por defecto).
- **command (función):** acepta una función a ser ejecutada cuando se presione el botón de guardado de valores. Su valor predeterminado es None. En el caso de tener el valor predeterminado, la función por defecto es gatherEntries (guarda todas las entradas en el atributo de instancia valores).
- **tituloGuardar (cadena):** define el título del botón que ejecutará la función en command. Su valor predeterminado es "Aceptar".
- **tituloBorrar (cadena):** define el título del botón que ejecutará la función deleteEntries (borra todas las entradas en el FieldFrame). Su valor predeterminado es "Aceptar".

Dentro de FieldFrame están definidos los siguientes métodos:

- **getValue:** recibe un criterio (cadena) de los que fueron usados para definir el objeto y devuelve su valor actual. Si el criterio no existe retorna None.
- **gatherEntries:** actualiza la lista de valores guardados a los últimos datos ingresados por el usuario. En caso de que alguna celda esté vacía, retornará una excepción del tipo errorEntradaNula.
- **deleteEntries:** elimina todas las entradas en el FieldFrame y deja las celdas vacías. Esta función es invocada cuando se presiona el botón borrar.

Los atributos que se definen dentro de la clase son:

- **Botones guardar y borrar:** al ser presionados, invocan la función en command y deleteEntries respectivamente.
- **Etiquetas tituloCriteriosWidget y tituloValoresWidget:** definen el título de cada columna del FieldFrame utilizando los valores de tituloCriterios y tituloValores.

Lugares donde se usa FieldFrame

```
3953 criterios = ["ID", "Nombre", "Tipo de artista"]
3954 valores = [f"{id_num}", "", ""] # valores vacíos inicialmente
3955 # Se crea el FieldFrame; al presionar "Guardar" se llamará a la función lambda
3956 # que invoca process_new_artist con los datos ingresados.
3957 ff = FieldFrame(process_frame,
3958                 tituloCriterios="Datos del nuevo artista",
3959                 criterios=criterios,
3960                 tituloValores="Ingrese valor",
3961                 valores=valores,
3962                 habilitado= ["Nombre", "Tipo de artista"],
3963                 combobox=False,
3964                 command=lambda: (process_new_artist(id_num, ff)))
3965 ff.pack(pady=10, fill="both", expand=True)
```

Datos del nuevo artista	Ingrese valor
ID	2
Nombre	
Tipo de artista	
Aceptar Borrar	

```

4147 week_days = Main.getWeek() # Retorna una lista de objetos date
4148 day_options = [day.strftime("%Y-%m-%d") for day in week_days]
4149
4150 # Crear un único FieldFrame para ingresar Día, Hora de inicio y Hora de fin.
4151 # Se pasan tres criterios y tres valores (el primero es una lista de opciones para el día).
4152 ff = FieldFrame(process_frame,
4153                 tituloCriterios="Programación de Clase",
4154                 criterios=["Día", "Inicio", "Fin"],
4155                 tituloValores="Valor",
4156                 valores=[day_options, "", ""],
4157                 combobox=False,
4158                 tituloGuardar="Programar",
4159                 command=lambda: (
4160                     process_schedule(actor, areaSeleccionada, nivelClase, ff))
4161                 )
4162 ff.pack(pady=10, fill="x")

```

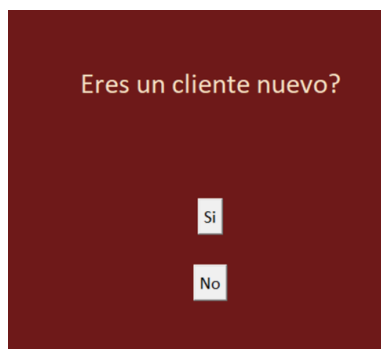
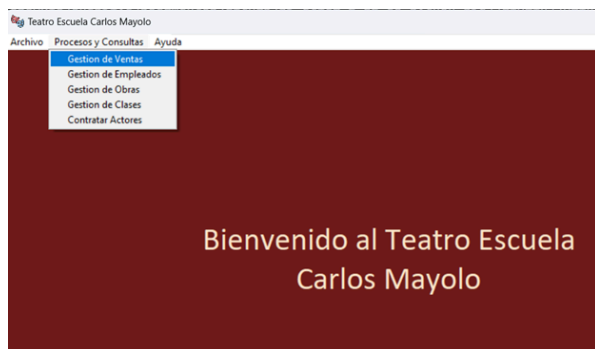
Programación de Clase	Valor
Día	▼
Inicio	
Fin	
Programar	
Borrar	

Hay muchísimos casos más, estos solo son algunos ejemplos puntuales

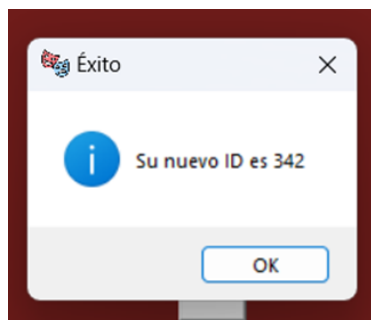
Manual de usuario

Funcionalidad 1

La función 1 de venta de tiquetes como dice el nombre maneja la venta de todos los productos del teatro, aquí vas a encontrar las suscripciones, sillas y las diferentes obras a comprar



Si el cliente es nuevo lo que hará será asignarle un número de identificación para que lo pueda usar la próxima vez que ingrese a el teatro



Si por el contrario ya tienes un numero de identificación válido, puedes usarlo para iniciar sesión y que toda la información antigua sea cargada, como el tipo de suscripción que tenías o la última función que compraste.

Lo siguiente será decidir si se quiere comprar una nueva suscripción, si eres nuevo tendras una suscripción básica por defecto, si decides adquirir una nueva suscripción esto te dará muchos beneficios

Una vez elijas tu suscripción de preferencia , el siguiente paso es elegir la Obra que desees ver, se te mostrará un menú con el precio y la duración de la función

Nombre	Genero	Duracion	Precio
El gran show	Circo	21015.0	\$13,200.00
Romeo y Julieta	Drama	21530.0	\$10,300.00
Cars 4	Terror	21015.0	\$10,200.00
El amante	Romance	21530.0	\$17,200.00
Nemo el musical	Musical	21530.0	\$10,300.00

Una vez seleccionada la obra que deseas se te presentarán las funciones que estén disponibles para esa obra en específico

Obras	Respuesta
FECHA 2025-02-24	HORA 15:00:00

Luego deberás escoger el tipo del asiento, para eso se te mostrará un gráfico representativo de cada silla y su ubicación en el teatro, además se mostrará el tipo de asiento, puesto que para poder acceder a ellas debes de tener la suscripción necesaria, y por último no podrás seleccionar las sillas que estén ocupadas

GO 0000	GO 0001	PR 0010	PR 0011	CO 0100	CO 0101	CO 0102	CO 0103
BA 1000	BA 1001	BA 1002	BA 1003	BA 1004	ocupado	BA 1006	BA 1007
BA 1008	BA 1009	BA 1010	BA 1011	BA 1012	BA 1013	BA 1014	BA 1015

Eleccion

seleccionaste las silla 101

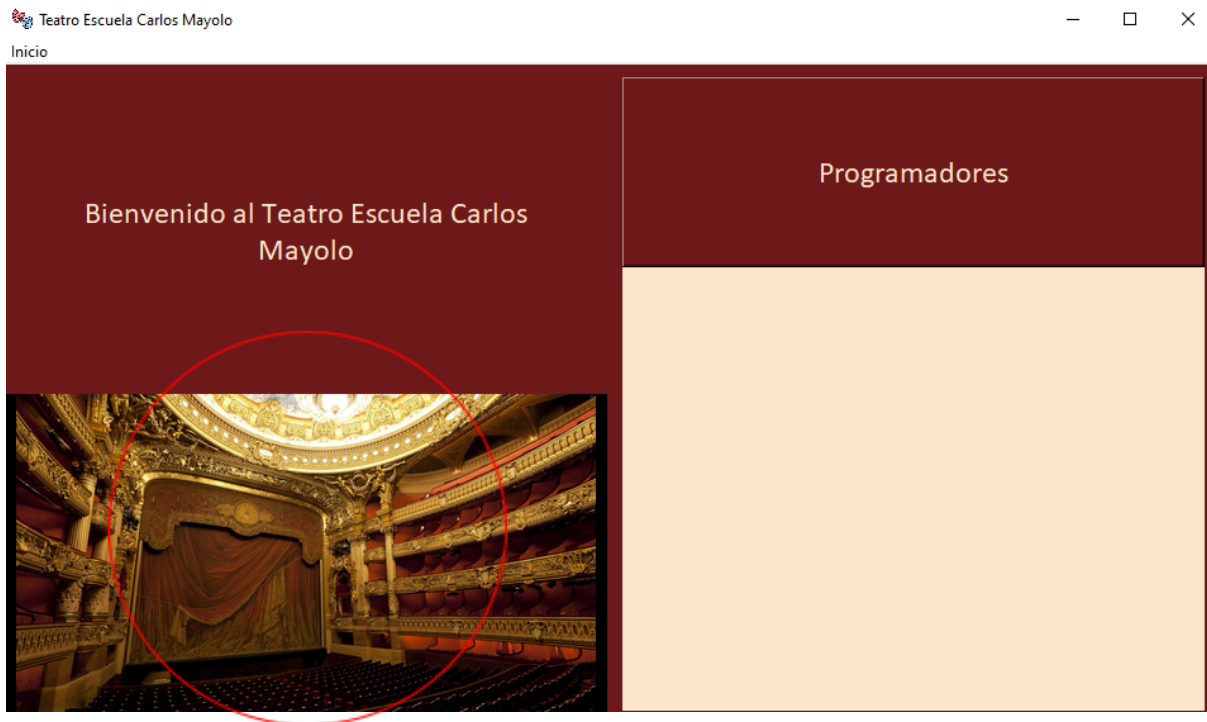
Yes No

Una vez seleccionado la silla lo único que queda es pagar, se re imprimira un ticket con tus compras y el valor de la misma

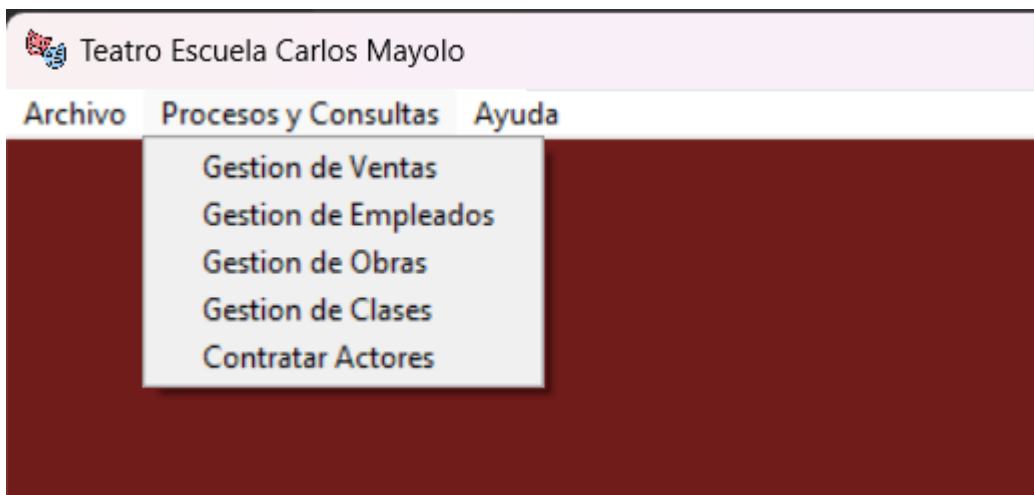
```
=====
                        Tiquete # 60
                        24/02/2025
=====
      Producto      Precio
-----
      Funcion      $9,180.00
      Suscripcion  $11,900.00
      Premium
      Descuento por ser Premium
                        -1,020.00
=====
                        GRACIAS POR SU COMPRA
=====
```

Funcionalidad 2

Para ingresar al sistema se debe clicar sobre las imágenes que rotan cuando pasa el mouse sobre ellas, ubicadas en la esquina inferior izquierda.



Al ingresar al sistema, para ejecutar la funcionalidad debemos clicar sobre el menú de Procesos y consultas, a continuación daremos click en el submenú Gestión de Empleados.



Al darle en procesos y consultas se obtiene el menú de las funcionalidades, en este caso vamos a ver el manual de usuario de Gestion Empleados.

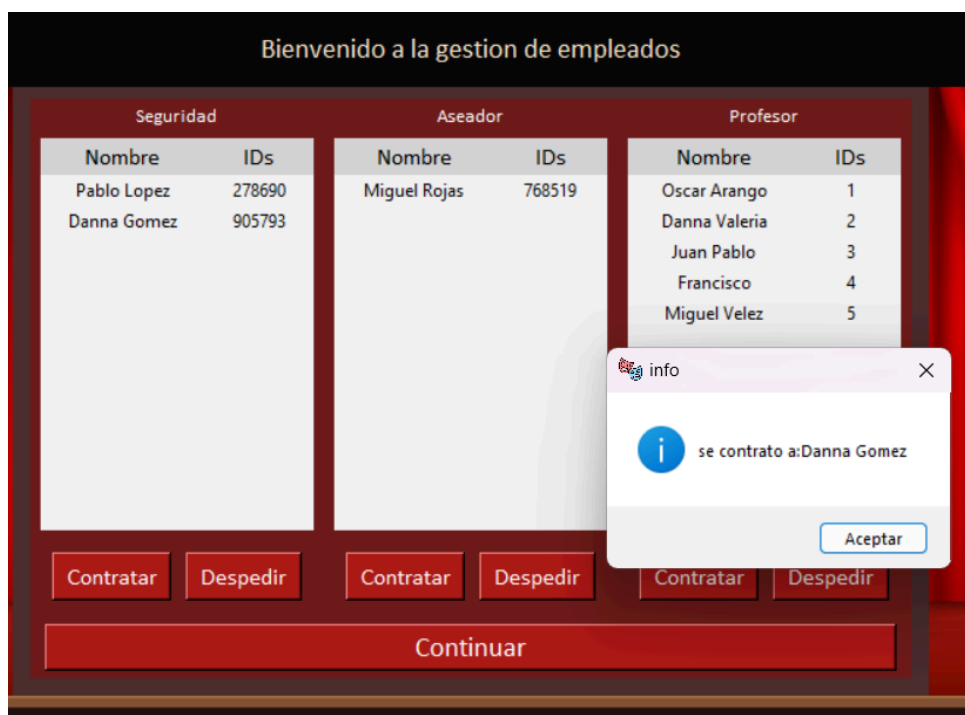
Nada más ingresar el sistema valida si hay deudas por pagar de los trabajadores y las paga automáticamente y luego imprime las listas de los trabajadores

Bienvenido a la gestion de empleados					
Seguridad		Aseador		Profesor	
Nombre	IDs	Nombre	IDs	Nombre	IDs
Pablo Lopez	278690	Miguel Rojas	768519	Oscar Arango	1
				Danna Valeria	2
				Juan Pablo	3
				Francisco	4
				Miguel Velez	5
Contratar	Despedir	Contratar	Despedir	Contratar	Despedir
Continuar					

Tenemos acciones para interactuar con estas listas, las cuales son contratar y despedir, en este ejemplo vamos a ver la opción de contratar, la cual es la misma interfaz para todos los botones de contratar



Entonces ingresando al botón de contratar en la tabla de seguridad, nos aparecerán 10 candidatos distintos, en el cual podemos elegir un candidato, dándole en el botón que aparece al lado derecho de cada uno, para ser prácticos elegiremos a Danna Gomez



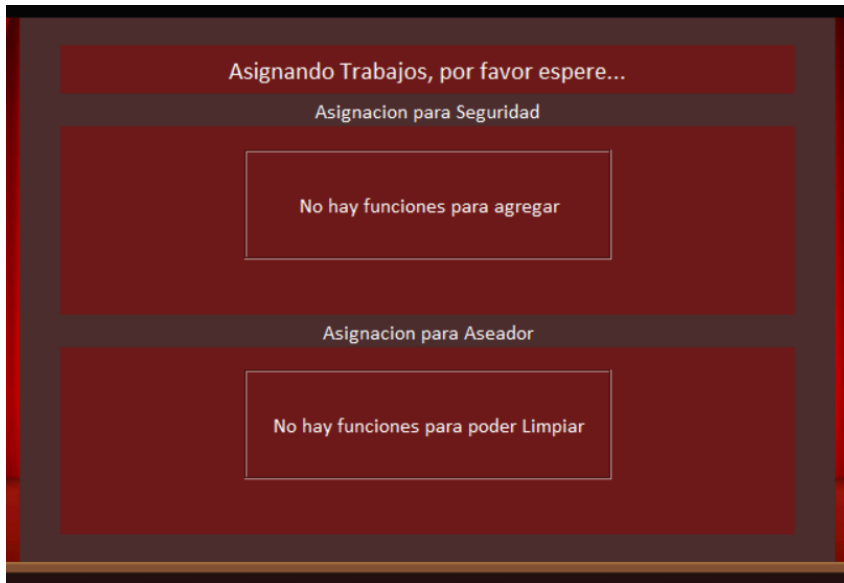
observamos que nos salta una ventana de diálogo que nos informa que el contrato se realizó de manera efectiva y se observa la actualización en la tabla correspondiente. Luego de esto daremos en despedir, la interfaz es la misma para todas las tablas; en este ejemplo despediremos al empleado que acabamos de contratar (Seguridad)



Se observa los 2 empleados disponibles de seguridad, procedemos a dar en el botón derecho de Danna Gomez para despedirla



vemos cómo se despidió correctamente a Danna y se le paga su liquidación, además la tabla de seguridad se actualizó y solo tenemos a un empleado. Daremos en el botón de continuar para empezar a asignar los trabajos. Aquí pueden ocurrir diferentes casos, o no se pueden asignar o se asignan correctamente



Aquí observamos como no se pudieron asignar trabajos correctamente a los empleados, por ende pasa a mostrar el ranking y a preguntar si desea realizar los pagos.





Por otro lado, cuando existen obras y trabajadores ,pasa a la asignación de trabajos y a su verificación, para luego mostrar el ranking y el panel para pagar a los empleados



En caso de no realizar los pagos, aparecera un boton de salir, el cual reiniciará la funcionalidad, en caso de realizarlos empezará a validar las cuentas a pagar e imprimirá los mensajes correspondientes y luego aparecera el boton de salir

El saldo de tesoreria es: \$10,000,000.00

Ranking Empleados

Nombre	IDs
Oscar Arango	1
Danna Valeria	2
Juan Pablo	3
Francisco	4
Miguel Velez	5
Pablo Lopez	278690
Miguel Rojas	768519

Pago exitoso

Se pago un total de: 0

Se realizo el pago a: 7 cuentas en total

El saldo de tesoreria es: \$10,000,000.00

Ranking Empleados

Nombre	IDs
Oscar Arango	1
Danna Valeria	2
Juan Pablo	3
Francisco	4
Miguel Velez	5
Pablo Lopez	278690
Miguel Rojas	768519

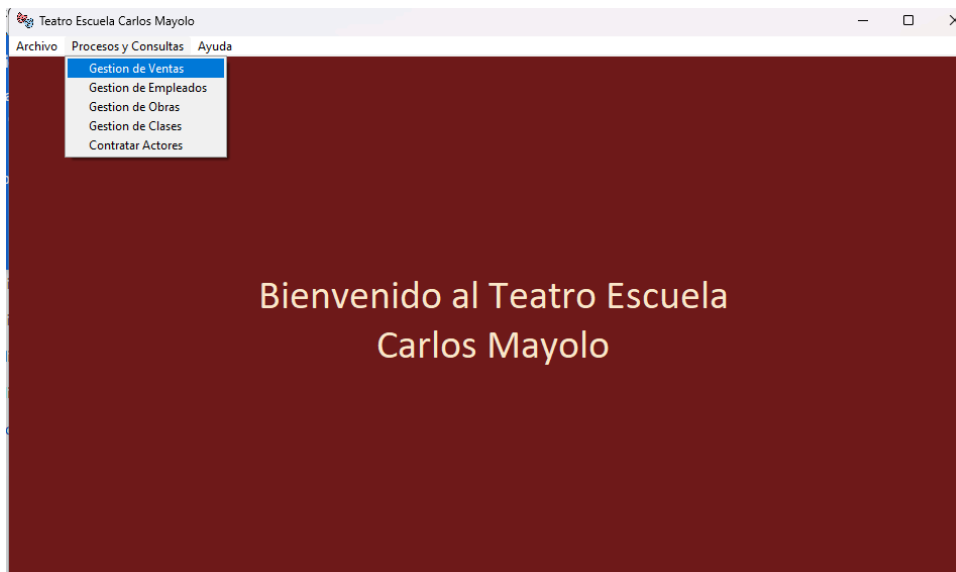
Salir

Funcionalidad 3

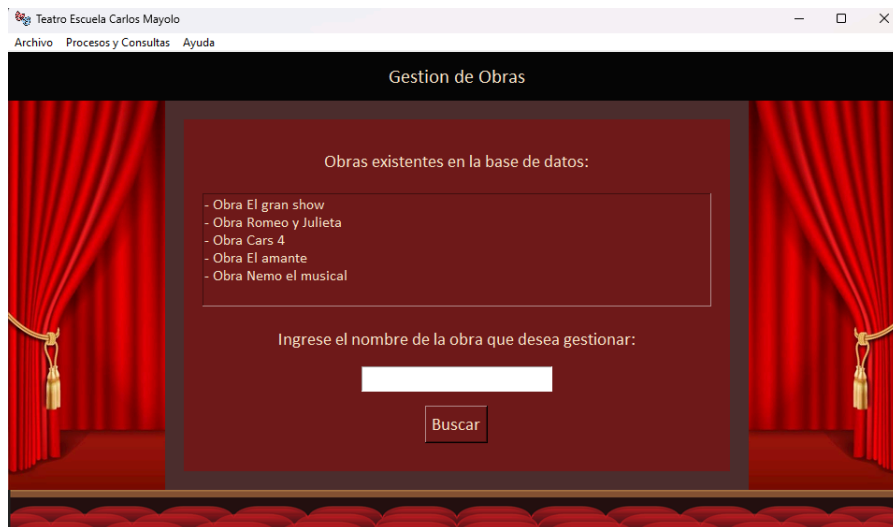
Para entrar a la ventana de funcionalidades debes acceder al bloque donde se enseñan las fotos (bloque inferior izquierdo)



Luego de entrar, serás enviado a esta ventana, donde debes seleccionar del menú, Procesos y Consultas, al seleccionarlo de la cascada debes seleccionar gestión de Obras



De ahí serás enviado a la pestaña de la funcionalidad, donde se te mostrarán las obras ya creadas, puedes añadir una nueva solo escribiendo el nombre de tu nueva obra.



Si decidiste crear una nueva obra, el programa te llevará a esta pestaña, en la el campo nombre ingresa el nombre de la obra, en el campo actor un actor previamente creado(puede consultarlos en la funcionalidad 4), en el campo aptitud la aptitud que es actor debe desempeñar (CANTO, IMPROVISACIÓN, BAILE, DISCURSO O EMOCIONALIDAD), en el campo genero añadir el género de la nueva obra (DRAMA, COMEDIA, MUSICAL, FANTASIA, TERROR, ROMANCE, CIRCO O EXPERIMENTAL), en el campo director añada un director asociado al género elegido, si el programa no encuentra su director será llevado a una pestaña donde podrá crearlo, finalice indicando datos numéricos asociados al costo de producción y a la duración en formato HHMMSS (ej. 011000 para 01h10m00s)

Datos de la nueva obra		Ingrese su respuesta <3
Nombre		te
Actor		Pedro Pineda
Aptitud		Canto
Genero		Comedia
Director		Luis Pineda
Costo de producción		10000
Duración de la obra (HHMMSS)		011000
Aceptar Cancelar		

si su director no se encontró será llevado a la pestaña de creación de directores, aquí ingrese el nombre del director y su id (el género será el que usted planeaba añadir a su obra. Una vez cree su director vuelva a hacer el proceso para llegar a la pestaña de creación de obras.

Datos del nuevo director

Ingrese su respuesta <3

Nombre

Luis Perez

ID

100000000

Genero

Genero COMEDIA

Aceptar

Borrar

Una vez su obra ha sido creada podemos continuar con la creación de funciones para ella, ingrese en número la cantidad de funciones que desea crear.

Teatro Escuela Carlos Mayolo

ArchivoProcesos y ConsultasAyuda

Gestion de Obras

Por favor digite:

Número de funciones

1

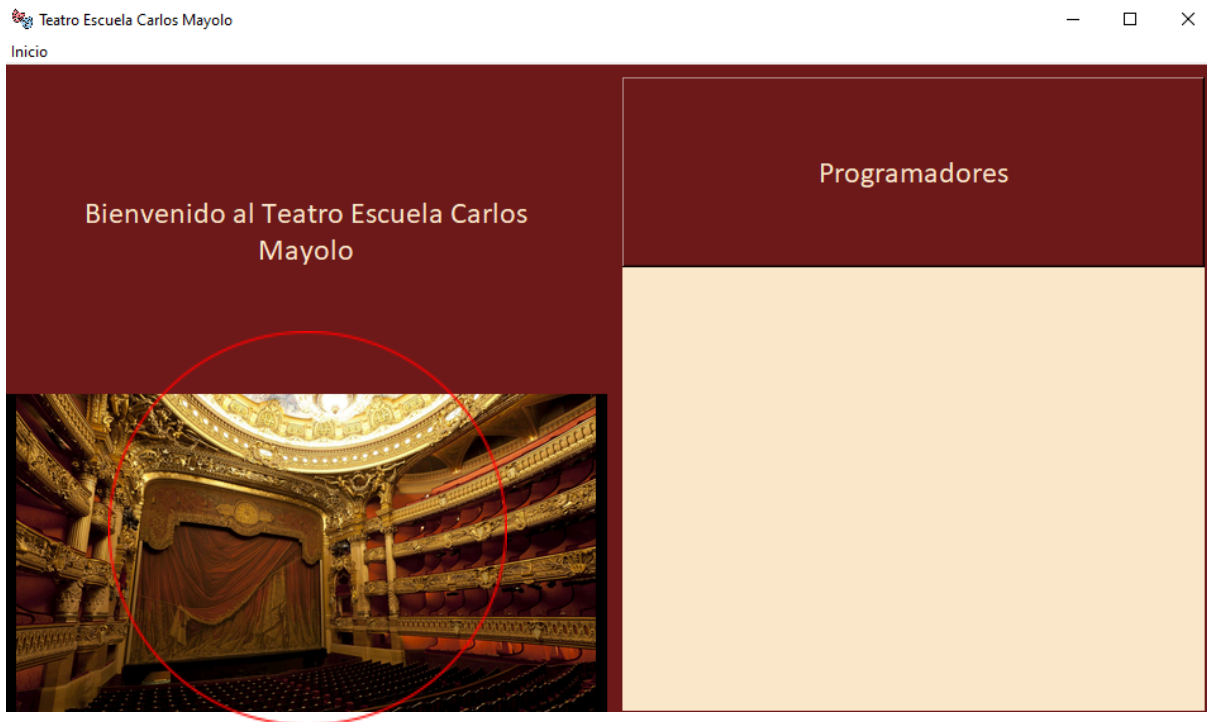
Aceptar

Borrar

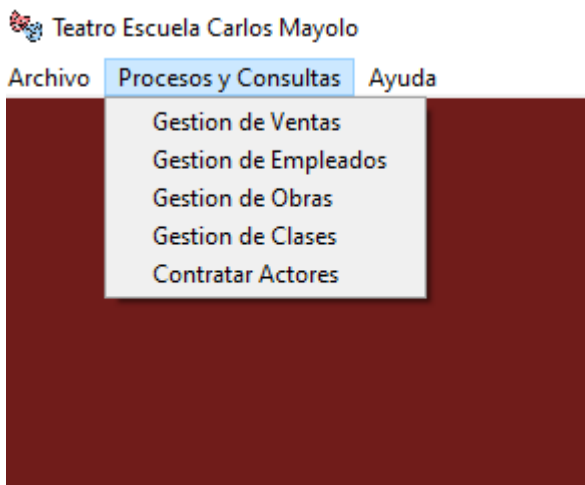
Al finalizar, se imprimirá la función creada y puede observar cómo se distribuyeron en el horario de cada sala.

Funcionalidad 4

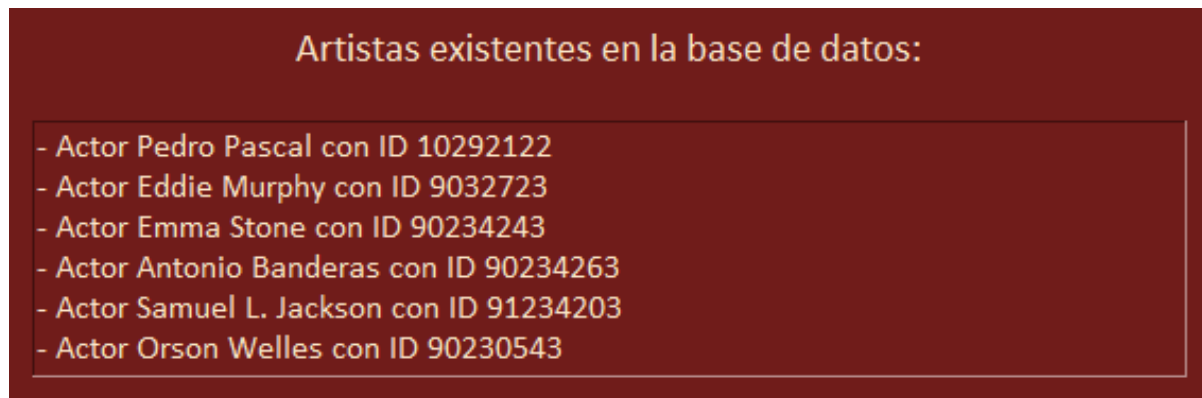
Para ingresar al sistema se debe clicar sobre las imágenes que rotan cuando pasa el mouse sobre ellas, ubicadas en la esquina inferior izquierda.



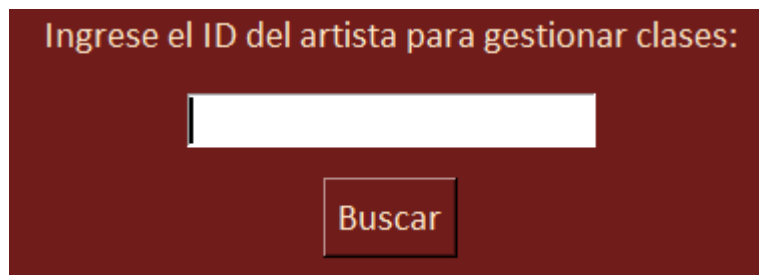
Al ingresar al sistema, para ejecutar la funcionalidad debemos clicar sobre el menú (Esquina superior izquierda) que dice “Procesos y consultas” y daremos click en el submenú “Gestion de Clases”.



Nada más ingresar, el sistema enseña una interfaz la cual muestra los artistas ya existentes en la base de datos, el cual podemos scrollear con el ratón para ver más información:



En esta misma interfaz el sistema pide al usuario digitar un ID. El ID debe ser un número entero, si no se cumple este requisito saldrá una advertencia):



Ingrese el ID del artista para gestionar clases:

Error



El ID debe ser un número entero.

Aceptar

Daremos click sobre el botón “Buscar”. Ahora bien, con esta interfaz tenemos dos opciones las cuales abordaremos ambas. Si decidimos escribir el ID de un actor no existente en la base de datos saldrá la siguiente interfaz:

Artista con ID 43 no encontrado.

¿Desea crear un nuevo Artista con este ID?

Sí

No

En caso de escoger la opción dos, se imprimirá lo siguiente, y para volver al inicio de la funcionalidad simplemente damos a “Volver al inicio”:

Obras en estado crítico del teatro:

- 'El gran show' (Promedio: 0)
- 'Romeo y Julieta' (Promedio: 0)
- 'Cars 4' (Promedio: 0)
- 'El amante' (Promedio: 0)
- 'Nemo el musical' (Promedio: 0)

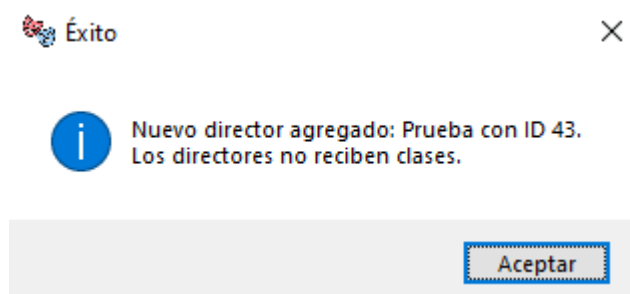
Volver al inicio

Si escogemos la opción 1. Se imprimirá lo siguiente:

Datos del nuevo artista	Ingrese valor
ID	43
Nombre	
Tipo de artista	
Aceptar Borrar	
(Si se ingresa 'actor' se pedirá la edad posteriormente)	

Aparecerá esta interfaz, en el apartado que está enfrente de “Nombre”, escribiremos caracteres alfanuméricos, es elección del usuario. Y en tipo de artista se deberá escribir “actor” o “director”. Si clickeamos en el botón “Borrar” se borrarán todas las entradas para volver a escribir. Al clickear en aceptar continuaremos a la siguiente interfaz.

Aquí, otra vez, tomaremos dos caminos. Si escribimos “director”, el programa mostrará una ventana emergente que dirá: Nuevo director agregado: {Nombre que escribimos previamente} con ID {ID que digitamos al principio del programa} y al dar click en “Aceptar” volverá al menú principal:



Si escogemos “actor”, el programa imprimirá nos pedirá que ingresemos la edad del actor, la edad debe estar entre los 4 y 80 años, si decidimos digitar una edad diferente se repetirá la pregunta hasta que digamos una edad válida mostrando advertencias en cada caso. Y daremos click en “Guardar”.

Ingrese la edad del nuevo actor (entre 4 y 80):

Guardar

Luego de esto, el sistema mostrará “Nuevo actor agregado” de forma similar a la interacción cuando creamos un director. Y además de eso, imprimirá una interfaz la cual da información relevante para el programa.#####

 Éxito

 Información

 Nuevo actor agregado: Test con ID 24 y edad 4.

 Se han inicializado las calificaciones del actor.

Aceptar

Aceptar

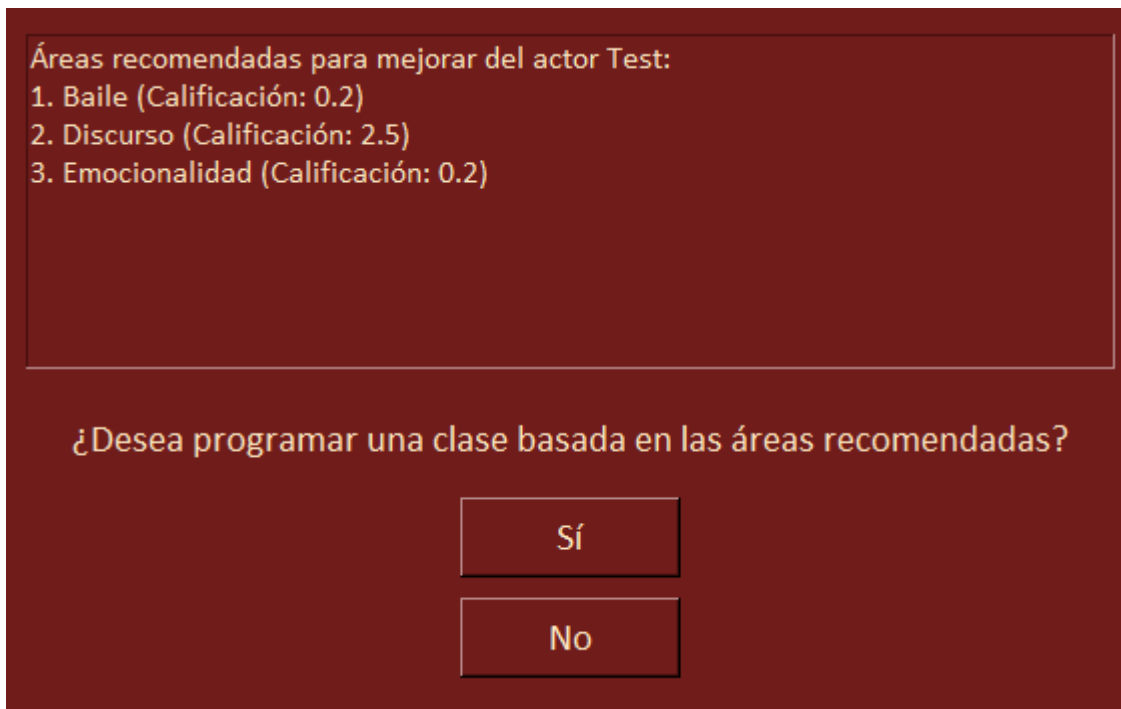
Calificaciones de profesores: [4.6, 0.2, 2.5, 0.2, 3.0]
Calificaciones del público: [0.5, 0.7, 2.0, 0.2, 3.1]

Obras en estado crítico del teatro:

- 'El gran show' (Promedio: 0)
- 'Romeo y Julieta' (Promedio: 0)
- 'Cars 4' (Promedio: 0)
- 'El amante' (Promedio: 0)
- 'Nemo el musical' (Promedio: 0)

Programar clase

Al dar click en “Programar clase” aparecerá la siguiente interfaz.



Áreas recomendadas para mejorar del actor Test:

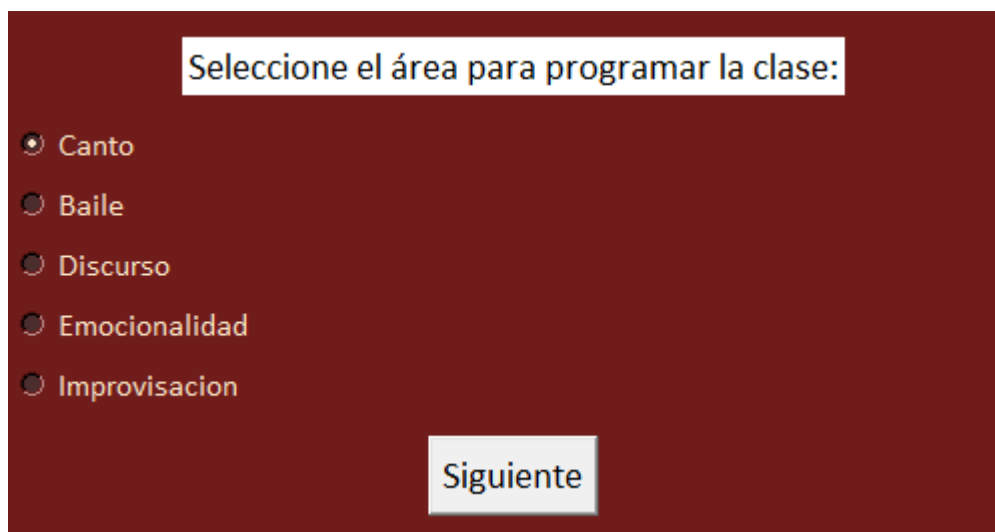
1. Baile (Calificación: 0.2)
2. Discurso (Calificación: 2.5)
3. Emocionalidad (Calificación: 0.2)

¿Desea programar una clase basada en las áreas recomendadas?

Sí

No

Si seleccionamos la opción “No”, nos aparecerá una interfaz donde escogemos la materia de la clase que deseamos dar al artista.



Seleccione el área para programar la clase:

- ☒ Canto
- ☐ Baile
- ☐ Discurso
- ☐ Emocionalidad
- ☐ Improvisacion

Siguiente

Y se continúa con el flujo de programar clases que se enseña a continuación.

Si se seleccionamos la opción 1 se escoge automáticamente el área con peor calificación:

Área seleccionada: Canto
Nivel de clase: Introducción

Programar la clase:

Programación de Clase	Valor
Día	
Inicio	
Fin	

Escogemos un día para programar la clase al dar click en la flecha que apunta hacia abajo. Y seguido nos pide poner el inicio del hora de la clase y el fin del horario de la clases (Este se escribe en formato 24 horas, del modo Hora:Minuto, ejemplo, si quiero ver una clase a la 1 pm, escribo 13:00. Las clases duran de 2 a 4 horas):

Seguido el programa imprime las siguientes interfaces, en las cuales solo hay que esperar.

Ahora aparecerá una interfaz con el fin de programar otra clase.

Y por último, aparece una interfaz en la cual se le dan puntos positivos al profesor quien dio la clase inicial al actor, para así volver al menú principal, o acabar con la funcionalidad. Ahora bien, ahí abarcamos todo lo que pasa cuando se escribe, en la interfaz donde se pregunta el ID del artista, un ID nuevo:

Ingrese el ID del artista para gestionar clases:

Si decidimos escribir un ID ya existente en la base de datos se ejecuta lo pertinente automáticamente, dependiendo si es actor o director. Si es actor, sucede todo lo que pasó desde la interacción con #####. Si es director, acaba la funcionalidad

Y así acabamos con el manual de usuario.

Funcionalidad 5

Al entrar a la funcionalidad será bienvenido con un menú para decidir si es cliente nuevo o existente, se debe introducir el número correspondiente a la opción preferida. Si tiene un ID es cliente existente, de lo contrario, cree un nuevo cliente.



Archivo Procesos y Consultas Ayuda

Bienvenido al panel de contratación de actores.

Requerimientos

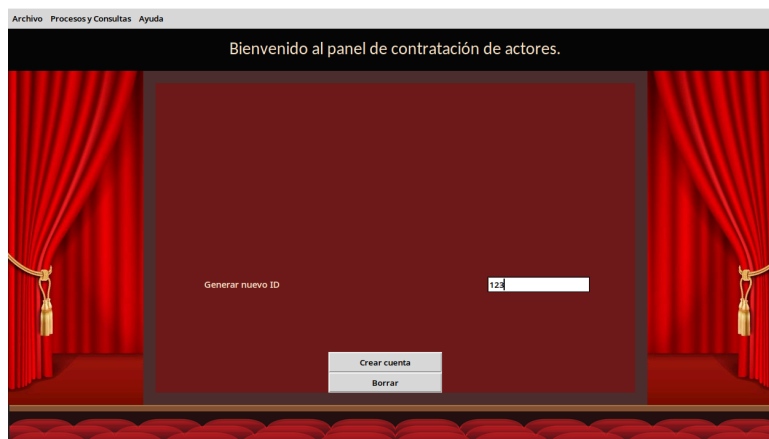
Por favor digite:

Tipo de Empresa

Aceptar

Borrar

Si es un cliente nuevo, debe crear un código de identificación con el cuál tendrá futuro acceso como cliente existente. Ambas opciones son para clientes de tipo Empresa. El código de identificación debe ser un entero positivo.



Archivo Procesos y Consultas Ayuda

Bienvenido al panel de contratación de actores.

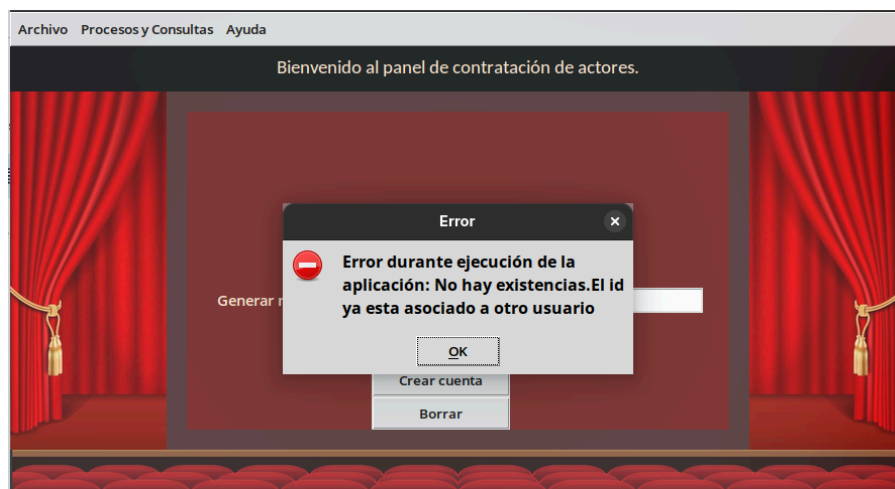
Generar nuevo ID

1234

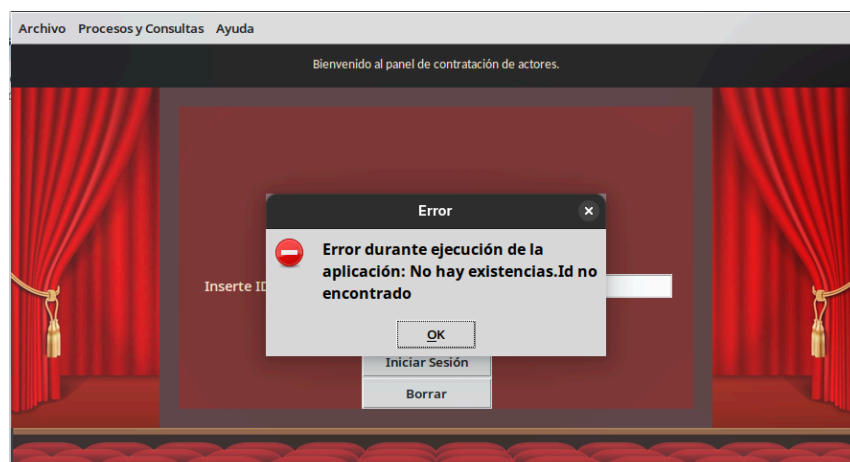
Crear cuenta

Borrar

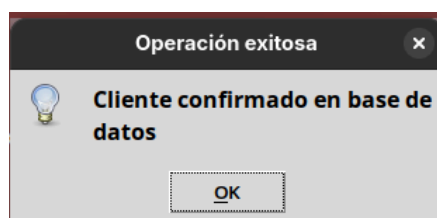
Es posible que el código de identificación ya exista, de ser así, se le señalará y deberá introducir un código distinto.



Si elige usar un cliente existente, e ingresa un código que aparece como cliente de tipo Persona y no tipo Empresa, también se señalará el error. De la misma forma aparecerá si no existe el código. Después del mensaje se retornará al menú principal.



De ser un código correcto ya registrado, se confirmará que se encontró.



Una vez el cliente esté configurado se harán preguntas sobre el actor a contratar, incluyendo el tipo de papel, tipo de obra para la que será contratado el actor, aptitud principal, y día de contratación.

The screenshot shows a software window titled "Bienvenido al panel de contratación de actores." (Welcome to the actor hiring panel). The window has a menu bar with "Archivo", "Procesos y Consultas", and "Ayuda". The main area is divided into two columns: "Características del actor" (Actor characteristics) and "Respuestas" (Responses). The "Características del actor" column contains four labels: "Tipo de papel", "Tipo de Obra", "Aptitud principal", and "Día de contratación". The "Respuestas" column contains four dropdown menus with the following selected values: "Rol principal", "Comedia", "Improvisacion", and "2025-02-25". At the bottom of the form are two buttons: "Continuar" (Continue) and "Borrar" (Delete). The interface is decorated with red curtains on the sides and a red carpet at the bottom.

Si resulta ser una entrada vacía, se mostrará el siguiente mensaje, para todo el código



Una vez elegida la fecha, es momento de decidir el inicio y fin del contrato en términos de horas, se debe introducir en formato 24 horas (Ejemplo: las dos de la tarde son las 14:00, las tres de la mañana son las 03:00). Además, se debe cumplir con los lineamientos de horarios para contratación: el mínimo de tiempo para contratar es de 4 horas, el máximo de 8 horas, y solo se puede contratar entre las 8 de la mañana y las 10 de la noche.

The screenshot shows a web application window titled "Bienvenido al panel de contratación de actores." (Welcome to the actor hiring panel). The interface has a dark red background with a stage curtain theme. It contains two input fields: "Hora de inicio" (Start time) with the value "12:00" and "Hora de fin" (End time) with the value "16:00". Below these fields are two buttons: "Continuar" (Continue) and "Borrar" (Clear). The top navigation bar includes links for "Archivo", "Procesos y Consultas", and "Ayuda".

Si el horario no cumple con los lineamientos postulados, se mostrará uno de los siguientes mensajes.

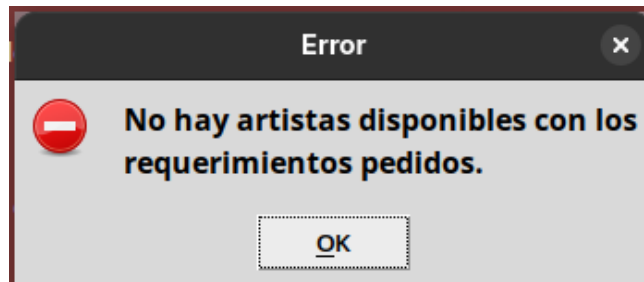
This is an error dialog box titled "Error". It contains a red circle with a white minus sign icon. The text reads: "La duración del horario escogido es incompatible con los lineamientos (entre 4 y 8 horas)". At the bottom, there is an "OK" button.

This is an error dialog box titled "Error". It contains a red circle with a white minus sign icon. The text reads: "Existe una incompatibilidad del horario con el lineamiento." followed by "Revise si:" and a list of two points: "1. El inicio del horario ocurre antes del fin del horario." and "2. Se exceden los límites de horario (muy temprano o muy tarde). Intente de nuevo." At the bottom, there is an "OK" button.

Se dará la opción de hacer búsqueda avanzada, que filtra por intervalo de edad y sexo.

This is a dialog box titled "Contratación de Actores". It features a question mark icon and the text "¿Desea hacer búsqueda avanzada?". At the bottom, there are two buttons: "Yes" and "No".

Si no se hace búsqueda avanzada y no se encuentran actores con esas características en el horario especificado, se retornará el siguiente mensaje.



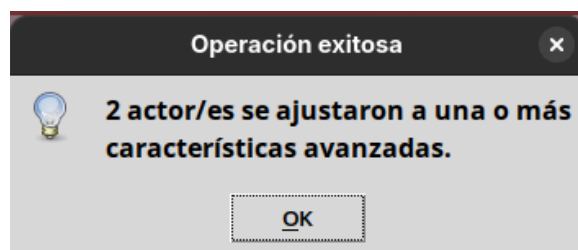
Si se decide hacer búsqueda avanzada, primero se preguntará el intervalo de edad en el que debe estar el actor, los intervalos son:

1. Infantil (menores de 15 años)
2. Juvenil (de edad mayor o igual a 15 años y menores de 25)
3. Adulto (de edad mayor o igual a 25 y menor a 70)
4. Adulto mayor (de edad mayor o igual a 70)

Luego se preguntará por el sexo que debe tener el actor.



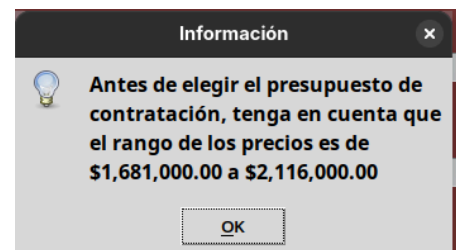
Si existen actores que cumplan al menos una de las características avanzadas, se retornará el mensaje.



De no ser así, se retornará el siguiente mensaje.



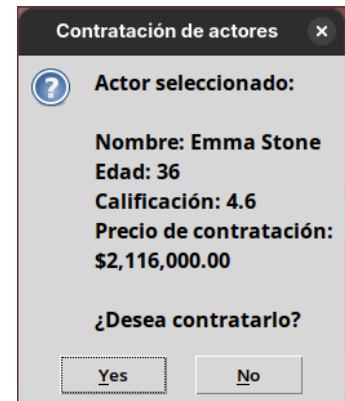
Una vez se tengan los actores filtrados, se pedirá el presupuesto de contratación, y se mostrará el rango de precios para los actores filtrados.



Si el presupuesto no puede cubrir ningún precio de contratación, se retornará un mensaje.



Si el presupuesto es suficiente para costear al menos un actor, se mostrará en un cuadro de lista los actores filtrados, una vez se elija uno, se abrirá una ventana de diálogo de confirmación donde se mostrará su nombre, precio de contratación, edad y calificación. Se debe recibir el dinero del cliente y luego presionar el actor a contratar, lo que retornará los detalles de la contratación y el horario, que será asignado al actor.



Si se confirma, se realiza la transacción y se actualiza el horario del actor, para así finalizar la funcionalidad con una ventana de confirmación.

